

1. Lctr 14–27 Recall Bullets**L14 Mixed C/ASM**

- EABI: R0–R3 args+ret, caller-saved. R4–R11 callee-saved.
- 64-bit arg/ret: low in R0, high in R1.
- Stack = Full-Descending: SP at last used; PUSH decrements then writes.
- Multi-push: low regs at low addrs regardless of brace order.
- Return: bx lr (no LR push) or pop {pc} (LR pushed).

L15 Arithmetic

- ADD/SUB/RSB (no S) $\nrightarrow$ flags. ADDS/SUBS $\rightarrow$ NZCV.
- ADC Rn,Op2: Rn+Op2+C. SBC Rn,Op2: Rn–Op2–(1–C).
- 64-bit add/adc. 64-bit sub/sbc.
- MUL/MLA/MLS: 32-bit. SMULL/UMULL/SHLLA/UMLAL: 64-bit.
- Modulo: udiv/mul/sub or udiv then mis Rd,Rn,Aq,Ra.

L16 Shifts / Op2

- LSL s&u. LSR u-only (0 fill). ASR s-only (sign repeat). S-shuf $\rightarrow$ C=last bit out.
- Op2 forms: #imm / Rm / Rm,{LSL/LSR/ASR/ROR} #n.
- Shifted-Op2: dest can NOT be omitted.
- Q31 mul (3-instr): smull r1,r0,r0,r1 / lsl r0,#1 / add r0,r0,r1,lsr #31.
- Fast-scale by $(2^n \pm 1)/2^n$: add r0,r0,r0,asr #n or sub ... asr #n.
- L17 Bitwise**
- AND/ORR/EOR/MVN. BIC = Rn & ~Op2. ORN = Rn | ~Op2. No C op for either.
- BFC Rd,#s,#w clears w bits at s. BFI inserts w LSBs.
- APSR: MRS Rd,apsr read; MSR apsr_nzcvq,Rn write.
- Bitfield assign: BIC mask, ORR value-shifted-in.

L18 Flags

- N=MSB, Z=(result==0), C=unsigned carry/|borrow, V=signed ovf.
- HW updates BOTH C and V every arithmetic; user picks.
- SUB: C=1 means NO borrow (u!>u2), C=0 means borrow.
- N-bit sim trick: shift operands left $32-N$, do ADDS, shift back ASR, read APSR.

L19 CCS

- CMP=SUB no-writeback. CMN=ADD no-writeback.
- cmp r0,#5 $\rightarrow$ assembler emits CMN r0,#5.
- Signed CCS: GT/GE/LT/LE. Unsigned: HI/HS/LO/LS. HS=CS, LO=CC.
- Match type to CCS: signed ops $\rightarrow$ signed CCS + ASR; unsigned $\rightarrow$ unsigned + LSR.

L20 LDR/STR (imm)

- [Ra,#os] basic; [Ra,#os]! update BEFORE; [Ra],#os update AFTER.
- LOAD sufs: —/B/SB/H/SH. STORE: —/B/H (no sign).
- C $\leftrightarrow$ ASM: *p++=post; ***p=pre; *(p+k)=basic.
- STRB writes only low byte of Rv; upper bits ignored.
- Offset ranges: word ± 4095 ; byte/half ± 255 on some forms.

L21 LDR/STR (reg) + MOV

- [Ra,Ri,LSL #n] with $n = \log_2(\text{elem size})$. No writeback form.
- MOV small imm. MOVW low16. MOVT high16.
- Pseudo ldr Rd,=expr picks MOV/MOVW/literal-pool by size.
- Two array iters: post-inc ptr vs fixed-base + reg-offset.

L22 Inc/Dec

- C precedence: postfix ++ L1 (LTR); prefix, *, &, cast L2 (RTL).
- *p++=deref old, advance. ***p=advance, deref new.
- *p++ = val++ in mem (ldr/add/str). ***p++ = post-inc p, store-back via [r0,#-1].
- Ptr arith scaled: p8+1, p16+2, p32+4. Typed diff $\rightarrow$ elements.

L23 ASM Calls ASM

- Leaf: no BL $\rightarrow$ bx lr, no LR push.
- Stem: has BL $\rightarrow$ push {lr} + pop {pc}.
- LDRD Rv,L,Rv,H,[Ra,#os]: 8 bytes, L=low addr, little-endian.
- No reg-offset form for LDRD/STRD.

L24 Combo Branch / CEX

- CBZ/CBNZ Rn,lbl: fwd only ($\leq 126B$), Rn=R0–R7, no flags.
- Perfect after LDRB (byte value already in reg, null-term test).
- IT/ITT/ITTE/ITTT/ITTEE/ITEE/ITTEE/ITTTT — up to 4 slots.
- First=T=cond; later T=cond, E=inverse.
- CBZ/CBNZ FORBIDDEN in IT. Cond B allowed ONLY as LAST slot.

L25 If/If-else

- PL: same-CCS branch to _then; else FIRST.
- NL: inverse-CCS branch to _else; then FIRST. Reads like C.
- if-elseif: chain CMP+NL-branch, b end each arm.
- Compound OR: double CMP $\rightarrow$ shared _then; fall-through=else.
- Compound AND (De Morgan): both CMPs $\rightarrow$ _else; fall-through=then.

L26 Loops

- Labels _lp (top), _end (exit). while/for top-tested (0+). do-while bottom-tested (≥ 1).

- while: NL cmp at top to _end, body, b lp.
- while-late: jump over body on entry, test at bottom.
- while-CEX (rare): cmp/it cond/bcond body_in_it/b_end.
- Priming-read duplicates LDRB; while(1)+break collapses.

L27 break/continue/switch

- break = fwd b_end. continue (while) = back to cond.
- continue in for = branch to UPDATE expr (i++ still runs).
- Switch naive: CMP/BEQ ladder; stacked labels = fall-through.
- ADDR Rd,lbl PC-rel addr; TBB [Rn,Rm]: PC=PC+4+2·table[Rm].
- TBB table bytes: (case_X-first_case)/2.

2. Q-Format Fixed-Point (HW5, L14)

UQm.n unsigned, $N = m + n$. Qm.n signed, $N = 1 + m + n$.

Encode: $I = \text{round}(f \cdot 2^n)$; if < 0 add 2^N , then hex.

Decode: if MSB=1 subtract 2^N (two's comp); $f = I/2^n$.

Format	Scale
Q15 / Q1.15	$2^{15} = 32768$
Q15.16	$2^{16} = 65536$
Q16.15	$2^{15} = 32768$ (32-bit signed)
Q3.4 (8-bit)	$2^4 = 16$
UQ3.5 (8-bit)	$2^5 = 32$

HW HW5 P4: encode 6.73 in Q3.4 $\rightarrow 6.73 \cdot 16 = 107.68 \rightarrow 108 = \mathbf{0x6C}$; encode $-4.5 \rightarrow -72 + 256 = 184 = \mathbf{0xB8}$.

HW HW5 P5 Q15 MAC: $f_1 f_2 + f_3 f_4$: encode each $(0.5 \rightarrow 0x4000, 0.25 \rightarrow 0x2000, 0.125 \rightarrow 0x1000, 0.0625 \rightarrow 0x0800)$. Product: $(I_A \cdot I_B) > > 15$. Sum results (plain int add).

Multiply rule: Qm.n $\times$ Qp.q = Qm+p+1.n+q (in 2N-bit). Shift right by n (or q) to renormalize to Qm.n (or Qp.q).

HW7 P1 Q15.16 extraction: after smull r1,r0,r0,r1 (R0=HW, R1=LW), result bits b47..b16:

```
smull r1,r0,r0,r1
lsl  r0,#16           @ HW << (47-31)
lsl  r0,r0,r1,lsr #16
Rule: lsl HW by (target_MSB-31); lsr LW by n. Q31: lsl1/lsr31. Q15.16: lsl16/lsr16. Q16.15: lsl17/lsr15.
```

3. Arithmetic Instrs (L14–15)

S-suffix rule: only S-variants update N,Z,C,V. ADD does NOT; ADDS does. MULS sets only N,Z (no C/V). UDIV/SDIV NEVER set flags (no S form).

ADDS/ADC — 64-bit add

```
@ R0=lo(A), R1=hi(A), R2=lo(B), R3=hi(B)
mp_add_64bit.s:
    add r0,r2      @ lo, sets C
    add r1,r3      @ hi + C
    bx  lr
```

HW HW6 P5: 0x88_000000+0x89_000000: lo=0x11_000000 C=1; hi=3+2+1=6.

SUBS/SBC — 64-bit sub

```
mp_sub_64bit.s:
    subs r0,r2      @ lo, sets C=NOT borrow
    sbc  r1,r3      @ hi - r3 - (1-C)
    bx  lr
```

ARM rule: C=1 $\Rightarrow$ NO borrow (i.e. $u_1 \geq u_2$); C=0 $\Rightarrow$ borrow. Opposite of x86.

Modulo via UDIV/MUL

$A \% B = A - (A/B) \cdot B$. Leaf, no push/pop:

```
mp_umod_32bit.s:
    udiv r2,r0,r1
    mul r2,r2,r1
    sub r0,r0,r2
    bx  lr
```

MUL alone (no S) never sets flags.

Signed/unsigned 64-bit products

Insn	Operation
UMULL Rlo,Rhi,Rn,Rm	unsigned {Rhi:Rlo} = Rn·Rm
SMULL Rlo,Rhi,Rn,Rm	signed {Rhi:Rlo} = Rn·Rm
UMLAL Rlo,Rhi,Rn,Rm	unsigned {Rhi:Rlo} += Rn·Rm
SMMLAL Rlo,Rhi,Rn,Rm	signed {Rhi:Rlo} += Rn·Rm
MLA Rd,Rn,Rm,Ra	Rd = Ra + Rn·Rm (32-bit)
MLS Rd,Rn,Rm,Ra	Rd = Ra - Rn·Rm (32-bit)

HW HW8 P4 (sum-of-squares, 64b):

```
ldr r2,#0
ldr r3,#0
lp:
    cmp r0,r1
    bgt end
    smlal r2,r3,r0,r0 @ {r3:r2}+=i*i
    add r0,r0,#1
    b  lp
end:
```

```
mov r0,r2
mov r1,r3
bx  lr
```

Return: R0=lo, R1=hi per AAPCS.

4. EABI Arg Passing (L15, HW6)

First 4 args $\rightarrow$ R0–R3, then stack. Sub-word args are promoted to 32-bit AT THE CALL SITE:

```
uint8_t 0xF  $\rightarrow$  R0 = 0x0000000F (zero-ext)
uint8_t 0xFF  $\rightarrow$  R1 = 0x000000FF (zero-ext)
int8_t 0x81 (= -127)  $\rightarrow$  R1 = 0xFFFFFFF81 (sign-ext)
int8_t 0x7E (= +126)  $\rightarrow$  R0 = 0x0000007E
```

Return: 32-bit in R0. 64-bit in R0:R1 (R0=lo). Sub-word returns go in R0 (implicitly extended by caller).

Q-format return: encode as described above, put hex into R0. Q15.16 of 3.25: $I = 3.25 \cdot 2^{16} = \mathbf{0x00034000}$.

5. Stack & PUSH/POP (L15)

ARM uses **full-descending** stack: SP points at top-most used word; PUSH decrements SP then writes.

PUSH ordering rule: Regardless of list syntax, push {list} writes lowest reg# at LOWEST address (ascending). Listing order is irrelevant.

HW HW6 P1: SP=0x20001020; push {r7,r8,r6,r5} $\rightarrow$ SP=0x20001010; storage: 0x...1010 R5, 0x...1014 R6, 0x...1018 R7, 0x...101C R8

Qz. Q3 P3: SP=0x20001020; push {r4,r8,r6} $\rightarrow$ SP=0x20001014; R4@1014, R6@1018, R8@101C. R6=(1<<<6)+(1<<8)=0x000000140; byte@1019 = second byte of R6 = 0x01 (little-endian).

push {r4-r7,lr}: SP=20; sorted low $\rightarrow$ hi. pop {r4-r7,pc}: restore and return (folds return into pop).

6. Barrel Shifter Op2 (L15–16)

Op2 forms: (a) #imm; (b) Rm; (c) Rm, {LSL|LSR|ASR|ROR} #n. Shift count MUST be imm in Op2 (standalone LSL/LSR can use Rm for count).

GNU quirk: with shifted-reg Op2, dest register cannot be omitted — add r0,r1,lsl #3 is illegal.

Shifted-Op2 scaling idioms

ASM	C equiv
lsl r0,r1,#n	$2^n \cdot r_1$ (s or u)
add r0,r1,r1,lsr #1	$3 \cdot r_1$
add r0,r1,r1,lsr #2	$5 \cdot r_1$
rsb r0,r1,r1,lsr #3	$7 \cdot r_1$
add r0,r1,r1,lsr #4	$17 \cdot r_1$
rsb r0,r1,r1,lsr #4	$15 \cdot r_1$
asr r0,r1,#n	signed $r_1/2^n$
lsr r0,r1,#n	unsigned $r_1/2^n$
add r0,r1,r1,asr #2	$1.25 \cdot r_1$ signed
sub r0,r1,r1,asr #2	$0.75 \cdot r_1$ signed
add r0,r1,r1,asr #3	$1.125 \cdot r_1$ signed
sub r0,r1,r1,asr #3	$0.875 \cdot r_1$ signed
rsb r0,r1,r1,asr #2	$-0.75 \cdot r_1$ signed

Rule: LSR $\Rightarrow$ unsigned only. ASR $\Rightarrow$ signed. LSL $\Rightarrow$ both. Decompose k as 1 ± 2^{-n} or $2^n \pm 1$ to fit one ADD/SUB/RSB.

Qz. Q3 P4 — $A = k \cdot B/8$ (signed, one instr):

- 6B/8: sub r0,r1,r1,asr #2 ($B - B/4$)
- 7B/8: sub r0,r1,r1,asr #3
- 9B/8: add r0,r1,r1,asr #3
- 10B/8: add r0,r1,r1,asr #2

S-suffix shifts & RRX

S-variants of LSL/LSR/ASR/ROR update C = last bit shifted out; also N,Z.

RRX Rd,Rn: 33-bit rotate right by 1; C $\rightarrow$ b31, b0 $\rightarrow$ (discarded unless RRRS). RRRS takes b0 $\rightarrow$ C.

C extraction trick: lsls r0,#1 puts old b31 into C. lsrs r0,#n puts old b(n–1) into C.

7. Bitwise Logic (L16–17)

Insn	Effect
AND Rd,Rn,Op2	$Rn \& Op2$
ORR Rd,Rn,Op2	$Rn Op2$ (set bits)
EOR Rd,Rn,Op2	$Rn \oplus Op2$ (toggle)
BIC Rd,Rn,Op2	$Rn \& \sim Op2$ (clear)
ORN Rd,Rn,Op2	$Rn \sim Op2$
MVN Rd,Op2	$\sim Op2$
BFC Rd,#s,#w	clear w bits at bit s
BFI Rd,Rn,#s,#w	insert Rn[w–1:0] into Rd[s+w–1:s]

BIC has no C-operator equivalent — must write a & -mask.

Bit-field idioms

Replace a nibble/field (“clear-then-insert”):

```
@ set bits[s+w-1:s] = v (w-bit)
bic r0,r0,mask,lsr #s @ clear
orr r0,r0,val,lsr #s @ insert
```

C form: a &= ~(mask<s); a |= (val & mask)<s;

Insert digit at an end (after shifting): the vacated nibble is already zero, so no BIC needed — just ORR.

HW HW7 P4: replace H_1 with 0xA in uint16_t a:

```
ldr r1,#15
bic r0,r0,r1,lsr #4
ldr r2,#10
orr r0,r0,r2,lsr #4
```

HW HW7 P5 (hand-eval): R0=0xAD, R1=0x03. bic r3,r0,r1=0xAC; orr r4,r3,r1=0xAF; eor r5,r0,r1=0xAE.

HW HW7 P6b (shift-left + insert low digit, $H_3 H_2 H_1 H_0 \rightarrow H_2 H_1 H_0 3$):

```
lsl r0,#4
orr r0,#3
```

P6c (shift-right + insert top digit, $\rightarrow H_3 H_2 H_1$):

```
lsr r0,#4
ldr r1,#7
lsl r1,r1,2
orr r0,r1
```

Qz. Q3 P5 — ASM uses bic...lsl #8+orr...lsl #8, mask=15: replace H_2 (bits 11:8) with low nibble of b. a &= ~(15<8); a |= (b & 15)<8;

8. Flags NZCV (L17–18)

APSR: N=b31, Z=b30, C=b29, V=b28 (Q=b27). **N,Z** used by both signed and unsigned. **C** = unsigned carry/borrow. **V** = signed overflow.

Processor sets ALL flags every arithmetic — it doesn't know signed vs unsigned. Programmer picks CCS.

Hand computation (N-bit system)

Let $C_N = 2^N$, $U_{\max} = 2^N - 1$, $S_{\min} = -2^{N-1}$, $S_{\max} = 2^{N-1} - 1$.

Per operation:

- Unsigned view: map negatives via $+C_N$, compute, reduce mod C_N . Cross-boundary $\Rightarrow$ C=1 (add) or C=0 (sub borrow).
- Signed view: compute directly. If outside $[S_{\min}, S_{\max}] \Rightarrow$ V=1, correct by $\pm C_N$.
- Binary result is the SAME for both views. N=MSB. Z=(result==0).

HW HW8 P1 (5-bit, $C_N = 32$) $-15 + (-5)$: U: $17+27=44-32=12$, C=1. S: $-20<-16$, $+32=12$, V=1. Binary 0b01100 (N=0, Z=0).

Qz. Q4 P2 (5-bit) 16–18: U: $16-18=-2+32=30$, borrow so C=0. S: $16-(-14)=30>15$ wait — actually 16 exceeds $S_{\max}$ itself so... the exam uses 16–18 with signed view: $-16-(-14)=-2 \in [-16,15]$, V=0. Binary 0b11110 (N=1, Z=0).

Mnemonic “C=|Borrow”: after subs: no borrow $\rightarrow$ C=1; borrow $\rightarrow$ C=0.

CMP / CMN / TST / TEQ

Insn	Op (discarded)	Flags
CMP Rn,Op2	$Rn - Op2$	N,Z,C,V
CMN Rn,Op2	$Rn + Op2$	N,Z,C,V
TST Rn,Op2	$Rn \& Op2$	N,Z,C
TEQ Rn,Op2	$Rn \& Op2$	N,Z,C

TST/TEQ do NOT affect V. CMP $\equiv$ SUBS-no-writeback; CMN $\equiv$ ADDS-no-writeback.

Pseudo: cmp r0,#5 assembles as CMN r0,#5. Signed CCS still valid.

APSR access: mrs r0,apsr (read); msr apsr_nzcvq,r0 (write).

9. CCS Table (L18–19)

CCS	S/U	Flags
EQ / NE	–	$Z=1 / Z=0$
GE	S	$V=N$
GT	S	$V=N \wedge Z=0$
LT	S	$V \neq N$
LE	S	$V \neq N \vee Z=1$
HS (CS)	U	$C=1$
LO (CC)	U	$C=0$
HI	U	$C=1 \wedge Z=0$
LS	U	$C=0 \vee Z=1$

	C	Signed	Unsigned
==	EQ	EQ	
!=	NE	NE	
>	GT	HI	
>=	GE	HS	
<	LT	LO	
<=	LE	LS	

Mnemonic “**HiLow unSun**”: Higher/Lower/Same = unsigned (can’t see the Sun above/below horizon).

Consistency rule: C-type determines **CCS family** (GT/GE/LT/LE vs HI/HS/LO/LS), **shift** (ASR vs LSR), and **load suffix** (SB/SH vs B/H).

HW HW8 P2: CMP with equal-magnitude diffs give identical flags because $-(-20) + 64 - ((-22) + 64) = 22 - 20$. CCS is **flag-driven**, not value-driven.

HW HW8 P3 (max via NL): $r_0 \geq r_1$?

```
cmp r0,r1
blt .else      @ !(a>=b)
bx lr          @ then: r0 is a
.else:
mov r0,r1
bx lr
```

10. LDR/STR Matrix (L20–21)

Two regs: Ra=addr, Rv=data.

LOAD suffixes			
Suf	Size	Ext	C type
—	word	—	(u)int32_t
B	ubyte	zero	uint8_t
SB	sbyte	sign	int8_t
H	half	zero	uint16_t
SH	half	sign	int16_t

STORE has NO sign variant — STR/STRH/STRB only.

Three addressing modes			
Mode	Effect		
[Ra,#os]	load from $Ra+os$; Ra unchanged		
[Ra,#os]!	$Ra+=os$, then load		
[Ra]#os	load, then $Ra+=os$		

Mnemonic: !=BEFORE; trailing #os=AFTER.

Register-offset (L21)			
ldr Rv,[Ra,Ri,LSL #2]	@ word, $Ra+4\cdot Ri$		
ldrbh Rv,[Ra,Ri,LSL #1]	@ half, $Ra+2\cdot Ri$		
ldrb Rv,[Ra,Ri]	@ byte, $Ra+Ri$		

Shift = log₂(elem size). No pre/post writeback form.

LDRD / STRD — 64-bit (L23)			
ldrd Rv_L,Rv_H,[Ra,#os]	@ basic		
ldrd Rv_L,Rv_H,[Ra,#os]!	@ pre		
ldrd Rv_L,Rv_H,[Ra],#os	@ post		

L=LSW (low addr), H=MSW, little-endian. **No reg-offset form**.

HW HW9 P1 (16 bytes 0x00..0x0F @ 0x20000000):
ldr r4,=0x20000000 → r4=base.
ldr r0,[r4,#4]: basic → r0=0x07060504.
ldr r1,[r4,#2]!: pre → r4=0x20000002, r1=0x05040302.
ldr r2,[r4]: post → r2=0x05040302, then r4=0x20000006.
ldr r3,[r4]: r3=0x09080706.

HW HW10 P2 T1: ldr r2,r3,[r0,#8]! on ipt[i]=i<<4 → r0+=8; r2=0xB0A09080, r3=0xF0E0D0C0.

11. C Pointer → ASM (L20, 22)

32-bit ptr: step=4			
C	LDR	STR	
*(p+k)	[r1,#4k]	[r1,#4k]	
++p	[r1,#4]!	[r1,#4]!	
*p++	[r1],#4	[r1],#4	
*p	[r1,#-4]!	[r1,#-4]!	
*p--	[r1],#-4	[r1],#-4	

16-bit signed: step=2 → LDRSH			
*(p+k) → ldrsh r0,[r1,#2k]			
++p → ldrsh r0,[r1,#2]!			
*p++ → ldrsh r0,[r1],#2			

Unsigned: LDRSH→LDRH. Stores: STRH (no sign).

8-bit: step=1 → LDRB/LDRSB
*(p+k) → ldrb r0,[r1,#k]
p++ → ldrb r0,[r1],#1
*p++ → ldrb r0,[r1],#1!

Array indexing (const ptr, index i)
ldr r4,[r0,r3,lsl #2] @ 32b: pArr[i]
ldrbh r4,[r0,r3,lsl #1] @ 16b
ldrb r4,[r0,r3] @ 8b

HW HW9 P2 mp_task1: ldrsh r2,[r0],#4 then str r2,[r1,#0]; *p32 = (int32_t)*p16s; p16s++2; (post-inc 4 = skip halfword).

HW HW9 P2 mp_task2: ldrh r2,[r0,#2]!: p16u++; *p32 = (int32_t)*p16u;.

HW HW10 P2 T4: ldrh r3,[r0,r2,LSL #1] with r2=1,2,3 → ipt[r1]=0x3020, ipt[r2]=0x5040. Register-offset = typed array indexing.

HW9 P3 four-task register state			
Entry: R0=ipt (0x20001010), R1=opt. Words: +0=0x30201000, +4=0x70605040, +8=0xB0A09080, +12=0xF0E0D0C0.			

Task	Form	R0 after	Printout
T1	basic [r0,#4] & #0xC	unchanged	0x70605040 / 0xF0E0D0C0
T2	pre-idx #4! ×2	0x...1018	0x70605040 / 0xB0A09080
T3	post-idx #4 ×2	0x...1018	0x30201000 / 0x70605040
T4	reg-off [r0,r2,LSL #2]	unchanged	0x70605040 / 0xB0A09080

C equiv (all use int32_t *p=(int32_t*)ipt):
• T1: opt[0]=p[1]; opt[1]=p[3];
• T2: p++; opt[0]=*p; p++; opt[1]=*p;
• T3: opt[0]=*p; p++; opt[1]=*p; p++;
• T4: int i=1; opt[0]=p[i]; i++; opt[1]=p[i]; i++;

HW10 P1 four-line memory trace				
uint8_t arr[i]=2*i @ 0x20000100. p8a=0x100, p8b=0x10C.				
Ln	C RHS	Mem Δ	p8a	p8b
4	*(p8a+2)	—	0x100	0x10D
5	++*(p8a+4)	[0x104]: 0x08→0x09	0x100	0x10E
6	++*p8a++	[0x100]: 0x00→0x01	0x101	0x10F
7	*p8a++	—	0x102	0x110

Final 4 bytes at 0x20000_10C: 0x04, 0x09, 0x01, 0x02.

HW10 P2 four-task LDRD/LDRSB/LDRSH trace			
ipt[i]=i<<4 @ 0x20001010. Entry R0=0x...1010.			

Task	ASM (1 key line)	Result
T1	ldrd r2,r3,[r0,#8]!	R0=0x...1018, r2=0xB0A09080, r3=0x00000004
T2	ldrbh r7,[r0,#4]! ×2	r2=0x000000040; r3=0xFFFFF80
T3	ldrbh ...,#10/#4 post	r2=0x00001000; r3=0xFFFFB0A0.
T4	ldrh r3,[r0,r2,LSL #1]	r2=1→3; r3=0x5040

Sign-extension pivots: byte 0x80 (b7=1); half 0x8000/0xB0A0 (b15=1). Byte 0x40: b7=0 → zero-like extension.

12. Sign-Extension Pitfalls

Insn	0x80 →	0xB0A0 →
LDRB	0x00000080	—
LDRSB	0xFFFFF80	—
LDRH	—	0x0000B0A0
LDRSH	—	0xFFFFB0A0

Check MSBit of loaded byte/half to predict extension.

HW HW10 P2 T2: ldrsb r2,[r0,#4]! at byte 0x40 → r2=0x000000040; next ldrsb at 0x80 → r3=0xFFFFF80.

HW HW10 P2 T3: ldrsh r3,[r0],#4 at halfword 0xB0A0 → r3=0xFFFFB0A0.

Qz. Q3 P6 (EABI sign-ext): int8_t 0x81 as fn arg → R1 =3; 0xFFFFF81. int8_t 0x7E → R0 = 0x0000007E.

13. Inc/Dec — Values vs Ptrs (L22)

C precedence: L1 (LTR): postfix +/--, [], (). L2 (RTL): prefix +/--, *, &, cast, sizeof.
p++ binds tighter than adjacent * or prefix ++, but its side effect (increment) happens **after** the original p is used.

Prefix vs postfix on a value
B[0] = ++A[0]; → inc first, then assign:
ldr r2,[r0] / add r2,#1
str r2,[r0] / str r2,[r1]
B[1] = A[1]++; → assign old, then inc:
ldr r2,[r0,#4] / str r2,[r1,#4]
add r2,#1 / str r2,[r0,#4]

Composite patterns			
C expr	ASM		
*p++	ldrb r2,[r0],#1		
*+p	ldrb r2,[r0],#1!		
*p++v	strb r2,[r0],#1		
*+p	ldrb / add#1 / strb		
*+p++	ldrb[r0],#1 / add / strb[r0,#-1]		
*p=*q+	ldrb [r1],#1 / strb [r0]		

HW HW10 P1 L5: *p8b++ = ++*(p8a+4); — inc byte at p8a+4 (0x08→0x09), write back, store:
ldrb r2,[r0,#4]
add r2,r2,#1
strb r2,[r0,#4] @ write-back!
strb r2,[r1],#1

HW HW10 P1 L6: *p8b++ = ++*p8a++;:
ldrb r2,[r0],#1 @ r0 now at old+1
add r2,r2,#1

strb r2,[r0,#-1] @ write back old addr
strb r2,[r1],#1
Ptr arithmetic: p8++→ +1, p16++→ +2, p32++→ +4. p-arr typed → elements; (void*)p-(void*)arr → bytes.

14. Function Calls (L23)

AAPCS regs:
R0–R3: args 1–4, ret (R0 or R1:R0); caller-saved.
R12 (IP): scratch; caller-saved.
R4–R11: general; **callee-saved** (push/pop if used).
LR: ret addr; save only if stem.
Leaf (no bl): bx lr; no LR push.
Stem: push {lr} + pop {pc}.
BL: bl tgt ⇒ LR=PC_next; PC=tgt.

Stem template
my_stem:
push {r4-r7,lr}
mov r4,r0 @ stash args
mov r5,r1
mov r6,r2
mov r7,r3
@ prep R0-R3, bl inner
pop {r4-r7,pc}

Load args by C type			
C type	Load		
(u)int32_t / * ldr			
int16_t	ldrbh		
uint16_t	ldrb		
int8_t/char	ldrbh		
uint8_t	ldrb		
(u)int64_t	ldrd rL,rH,...		

Return: ≤ 32b → STR/STRH/STRB on R0. 64b → strd r0,r1,...
Array-loop + call
ldrb r7,r0!
ldrb r8,r0!
ldrb r9,r0!
ldrb r10,r0!
ldrb r11,r0!
ldrb r12,r0!
ldrb r13,r0!
ldrb r14,r0!
ldrb r15,r0!
ldrb r16,r0!
ldrb r17,r0!
ldrb r18,r0!
ldrb r19,r0!
ldrb r20,r0!
ldrb r21,r0!
ldrb r22,r0!
ldrb r23,r0!
ldrb r24,r0!
ldrb r25,r0!
ldrb r26,r0!
ldrb r27,r0!
ldrb r28,r0!
ldrb r29,r0!
ldrb r30,r0!
ldrb r31,r0!
ldrb r32,r0!
ldrb r33,r0!
ldrb r34,r0!
ldrb r35,r0!
ldrb r36,r0!
ldrb r37,r0!
ldrb r38,r0!
ldrb r39,r0!
ldrb r40,r0!
ldrb r41,r0!
ldrb r42,r0!
ldrb r43,r0!
ldrb r44,r0!
ldrb r45,r0!
ldrb r46,r0!
ldrb r47,r0!
ldrb r48,r0!
ldrb r49,r0!
ldrb r50,r0!
ldrb r51,r0!
ldrb r52,r0!
ldrb r53,r0!
ldrb r54,r0!
ldrb r55,r0!
ldrb r56,r0!
ldrb r57,r0!
ldrb r58,r0!
ldrb r59,r0!
ldrb r60,r0!
ldrb r61,r0!
ldrb r62,r0!
ldrb r63,r0!
ldrb r64,r0!
ldrb r65,r0!
ldrb r66,r0!
ldrb r67,r0!
ldrb r68,r0!
ldrb r69,r0!
ldrb r70,r0!
ldrb r71,r0!
ldrb r72,r0!
ldrb r73,r0!
ldrb r74,r0!
ldrb r75,r0!
ldrb r76,r0!
ldrb r77,r0!
ldrb r78,r0!
ldrb r79,r0!
ldrb r80,r0!
ldrb r81,r0!
ldrb r82,r0!
ldrb r83,r0!
ldrb r84,r0!
ldrb r85,r0!
ldrb r86,r0!
ldrb r87,r0!
ldrb r88,r0!
ldrb r89,r0!
ldrb r90,r0!
ldrb r91,r0!
ldrb r92,r0!
ldrb r93,r0!
ldrb r94,r0!
ldrb r95,r0!
ldrb r96,r0!
ldrb r97,r0!
ldrb r98,r0!
ldrb r99,r0!
ldrb r100,r0!
ldrb r101,r0!
ldrb r102,r0!
ldrb r103,r0!
ldrb r104,r0!
ldrb r105,r0!
ldrb r106,r0!
ldrb r107,r0!
ldrb r108,r0!
ldrb r109,r0!
ldrb r110,r0!
ldrb r111,r0!
ldrb r112,r0!
ldrb r113,r0!
ldrb r114,r0!
ldrb r115,r0!
ldrb r116,r0!
ldrb r117,r0!
ldrb r118,r0!
ldrb r119,r0!
ldrb r120,r0!
ldrb r121,r0!
ldrb r122,r0!
ldrb r123,r0!
ldrb r124,r0!
ldrb r125,r0!
ldrb r126,r0!
ldrb r127,r0!
ldrb r128,r0!
ldrb r129,r0!
ldrb r130,r0!
ldrb r131,r0!
ldrb r132,r0!
ldrb r133,r0!
ldrb r134,r0!
ldrb r135,r0!
ldrb r136,r0!
ldrb r137,r0!
ldrb r138,r0!
ldrb r139,r0!
ldrb r140,r0!
ldrb r141,r0!
ldrb r142,r0!
ldrb r143,r0!
ldrb r144,r0!
ldrb r145,r0!
ldrb r146,r0!
ldrb r147,r0!
ldrb r148,r0!
ldrb r149,r0!
ldrb r150,r0!
ldrb r151,r0!
ldrb r152,r0!
ldrb r153,r0!
ldrb r154,r0!
ldrb r155,r0!
ldrb r156,r0!
ldrb r157,r0!
ldrb r158,r0!
ldrb r159,r0!
ldrb r160,r0!
ldrb r161,r0!
ldrb r162,r0!
ldrb r163,r0!
ldrb r164,r0!
ldrb r165,r0!
ldrb r166,r0!
ldrb r167,r0!
ldrb r168,r0!
ldrb r169,r0!
ldrb r170,r0!
ldrb r171,r0!
ldrb r172,r0!
ldrb r173,r0!
ldrb r174,r0!
ldrb r175,r0!
ldrb r176,r0!
ldrb r177,r0!
ldrb r178,r0!
ldrb r179,r0!
ldrb r180,r0!
ldrb r181,r0!
ldrb r182,r0!
ldrb r183,r0!
ldrb r184,r0!
ldrb r185,r0!
ldrb r186,r0!
ldrb r187,r0!
ldrb r188,r0!
ldrb r189,r0!
ldrb r190,r0!
ldrb r191,r0!
ldrb r192,r0!
ldrb r193,r0!
ldrb r194,r0!
ldrb r195,r0!
ldrb r196,r0!
ldrb r197,r0!
ldrb r198,r0!
ldrb r199,r0!
ldrb r200,r0!
ldrb r201,r0!
ldrb r202,r0!
ldrb r203,r0!
ldrb r204,r0!
ldrb r205,r0!
ldrb r206,r0!
ldrb r207,r0!
ldrb r208,r0!
ldrb r209,r0!
ldrb r210,r0!
ldrb r211,r0!
ldrb r212,r0!
ldrb r213,r0!
ldrb r214,r0!
ldrb r215,r0!
ldrb r216,r0!
ldrb r217,r0!
ldrb r218,r0!
ldrb r219,r0!
ldrb r220,r0!
ldrb r221,r0!
ldrb r222,r0!
ldrb r223,r0!
ldrb r224,r0!
ldrb r225,r0!
ldrb r226,r0!
ldrb r227,r0!
ldrb r228,r0!
ldrb r229,r0!
ldrb r230,r0!
ldrb r231,r0!
ldrb r232,r0!
ldrb r233,r0!
ldrb r234,r0!
ldrb r235,r0!
ldrb r236,r0!
ldrb r237,r0!
ldrb r238,r0!
ldrb r239,r0!
ldrb r240,r0!
ldrb r241,r0!
ldrb r242,r0!
ldrb r243,r0!
ldrb r244,r0!
ldrb r245,r0!
ldrb r246,r0!
ldrb r247,r0!
ldrb r248,r0!
ldrb r249,r0!
ldrb r250,r0!
ldrb r251,r0!
ldrb r252,r0!
ldrb r253,r0!
ldrb r254,r0!
ldrb r255,r0!
ldrb r256,r0!
ldrb r257,r0!
ldrb r258,r0!
ldrb r259,r0!
ldrb r260,r0!
ldrb r261,r0!
ldrb r262,r0!
ldrb r263,r0!
ldrb r264,r0!
ldrb r265,r0!
ldrb r266,r0!
ldrb r267,r0!
ldrb r268,r0!
ldrb r269,r0!
ldrb r270,r0!
ldrb r271,r0!
ldrb r272,r0!
ldrb r273,r0!
ldrb r274,r0!
ldrb r275,r0!
ldrb r276,r0!
ldrb r277,r0!
ldrb r278,r0!
ldrb r279,r0!
ldrb r280,r0!
ldrb r281,r0!
ldrb r282,r0!
ldrb r283,r0!
ldrb r284,r0!
ldrb r285,r0!
ldrb r286,r0!
ldrb r287,r0!
ldrb r288,r0!
ldrb r289,r0!
ldrb r290,r0!
ldrb r291,r0!
ldrb r292,r0!
ldrb r293,r0!
ldrb r294,r0!
ldrb r295,r0!
ldrb r296,r0!
ldrb r297,r0!
ldrb r298,r0!
ldrb r299,r0!
ldrb r300,r0!
ldrb r301,r0!
ldrb r302,r0!
ldrb r303,r0!
ldrb r304,r0!
ldrb r305,r0!
ldrb r306,r0!
ldrb r307,r0!
ldrb r308,r0!
ldrb r309,r0!
ldrb r310,r0!
ldrb r311,r0!
ldrb r312,r0!
ldrb r313,r0!
ldrb r314,r0!
ldrb r315,r0!
ldrb r316,r0!
ldrb r317,r0!
ldrb r318,r0!
ldrb r319,r0!
ldrb r320,r0!
ldrb r321,r0!
ldrb r322,r0!
ldrb r323,r0!
ldrb r324,r0!
ldrb r325,r0!
ldrb r326,r0!
ldrb r327,r0!
ldrb r328,r0!
ldrb r329,r0!
ldrb r330,r0!
ldrb r331,r0!
ldrb r332,r0!
ldrb r333,r0!
ldrb r334,r0!
ldrb r335,r0!
ldrb r336,r0!
ldrb r337,r0!
ldrb r338,r0!
ldrb r339,r0!
ldrb r340,r0!
ldrb r341,r0!
ldrb r342,r0!
ldrb r343,r0!
ldrb r344,r0!
ldrb r345,r0!
ldrb r346,r0!
ldrb r347,r0!
ldrb r348,r0!
ldrb r349,r0!
ldrb r350,r0!
ldrb r351,r0!
ldrb r352,r0!
ldrb r353,r0!
ldrb r354,r0!
ldrb r355,r0!
ldrb r356,r0!
ldrb r357,r0!
ldrb r358,r0!
ldrb r359,r0!
ldrb r360,r0!
ldrb r361,r0!
ldrb r362,r0!
ldrb r363,r0!
ldrb r364,r0!
ldrb r365,r0!
ldrb r366,r0!
ldrb r367,r0!
ldrb r368,r0!
ldrb r369,r0!
ldrb r370,r0!
ldrb r371,r0!
ldrb r372,r0!
ldrb r373,r0!
ldrb r374,r0!
ldrb r375,r0!
ldrb r376,r0!
ldrb r377,r0!
ldrb r378,r0!
ldrb r379,r0!
ldrb r380,r0!
ldrb r381,r0!
ldrb r382,r0!
ldrb r383,r0!
ldrb r384,r0!
ldrb r385,r0!
ldrb r386,r0!
ldrb r387,r0!
ldrb r388,r0!
ldrb r389,r0!
ldrb r390,r0!
ldrb r391,r0!
ldrb r392,r0!
ldrb r393,r0!
ldrb r394,r0!
ldrb r395,r0!
ldrb r396,r0!
ldrb r397,r0!
ldrb r398,r0!
ldrb r399,r0!
ldrb r400,r0!
ldrb r401,r0!
ldrb r402,r0!
ldrb r403,r0!
ldrb r404,r0!
ldrb r405,r0!
ldrb r406,r0!
ldrb r407,r0!
ldrb r408,r0!
ldrb r409,r0!
ldrb r410,r0!
ldrb r411,r0!
ldrb r412,r0!
ldrb r413,r0!
ldrb r414,r0!
ldrb r415,r0!
ldrb r416,r0!
ldrb r417,r0!
ldrb r418,r0!
ldrb r419,r0!
ldrb r420,r0!
ldrb r421,r0!
ldrb r422,r0!
ldrb r423,r0!
ldrb r424,r0!
ldrb r425,r0!
ldrb r426,r0!
ldrb r427,r0!
ldrb r428,r0!
ldrb r429,r0!
ldrb r430,r0!
ldrb r431,r0!
ldrb r432,r0!
ldrb r433,r0!
ldrb r434,r0!
ldrb r435,r0!
ldrb r436,r0!
ldrb r437,r0!
ldrb r438,r0!
ldrb r439,r0!
ldrb r440,r0!
ldrb r441,r0!
ldrb r442,r0!
ldrb r443,r0!
ldrb r444,r0!
ldrb r445,r0!
ldrb r446,r0!
ldrb r447,r0!
ldrb r4

```
    @ body
ldrb r2,[r1],#1    @ in-loop (DUP)
    lp
end:
```

do-while (bottom-tested)

```
lp:
    @ body
cmp r0,r1
blt lp
```

for loop

```
mov r4,#0
lp:
    cmp r4,r5
bge end
    @ body
add r4,#1
    b lp
end:
```

while(1) + break (no priming dup)

```
lp:
    ldrb r2,[r1],#1    @ SINGLE read
    cbz r2,end
    @ body
    b lp
end:
```

HW HW12 P2 P1 (plain while, lc→UC copy): priming LDRB + in-loop LDRB with sub r2,#0x20 between; terminator stored at _end.

HW HW12 P2 P3 (while(1)+break): single in-loop LDRB:

```
lp:
    ldrb r2,[r1],#1
    cbz r2,end
    sub r2,r2,#0x20
    strb r2,[r0],#1
    b lp
end:
    strb r2,[r0],#1    @ '0' store
    bx lr
```

HW HW12 P1 (swap-last-2-of-3): r12=pair counter (scratch, no push):

```
mov r12,#0
lp:
    ldrb r1,[r0],#1
    cbz r1,end
    ldrb r2,[r0],#1
    cbz r2,end
    ldrb r3,[r0],#1
    cbz r3,end
    strb r3,[r0,#-2] @ *(s-2)=c3
    strb r2,[r0,#-1] @ *(s-1)=c2
    add r12,r12,#1
    b lp
end:
```

```
mov r0,r12
bx lr
```

3 CBZ after 3 post-index LDRB catches early null.

while-using-CEX (rare)

```
lp:
    cmp r0,r1
    it lt
    blt body_in_it    @ last slot
    b end
body_in_it:
    @ body
    b lp
end:
```

18. break, continue, switch (L27)

- break = forward b end (or inline CBZ/CBNZ).
- continue (while/do-while) = back b lp.
- continue in for = branch to **update**, not top.
- CBZ/CBNZ: no IT; free-standing preferred null-test.

switch: CMP/BEQ ladder

```
cmp r0,#65    @ 'A'
beq case_a
cmp r0,#66
beq case_b
b default_lbl
case_a:
case_b:    @ stacked=fall-through
    ldr r0,#80
    b end
default_lbl:
    ldr r0,#69
end:
    bx lr
```

switch: ADR+TBB (O(1))

```
adr r1,table
tbb [r1,r0]    @ PC=PC+4+2*table[r0]
case_a:
...
```

```
table:
    .byte 0
    .byte (case_b-case_a)/2
```

19. Quiz 3 Problems (ASM in C)

Qz. Q3 P1 Return signed -2 from int32 fn: return reg R0 = 0xFFFFFFFEE (two's comp). Not 0x80000002.

Qz. Q3 P2 64-bit add test design: pick A, B whose **low 32b** sum crosses 2^{32} for one case and doesn't for the other. Shifting a nibble into low-word MSB flips C. A1=0x377<<24, B1=0x287<<24 → C=0. A2=0x378<<24, B2=0x288<<24 → C=1.

Qz. Q3 P3 push {r4,r8,r6}: SP=12; R4@..14, R6@..18, R8@..1C. Byte@..19 = 2nd byte of R6 little-endian.

Qz. Q3 P4 $A = k \cdot B/8$ single-instr w/ ASR Op2 (see Barrel Shifter sect).

Qz. Q3 P5 BIC/ORR bitfield splice ↔ C: ASM bic r0,r0,#15,ls1 #8 + orr r0,r0,(b&15),ls1 #8 = a & ~-(15<<8); a |= (b&15)<<8;

Qz. Q3 P6 sign-ext at call: int8_t 0x7E→R0=0x7E; int8_t 0x81→R1=0xFFFFFFF81.

20. Quiz 4 Problems (CCS+LDR/STR)

Qz. Q4 P1 if (a>0) return x; else return y;

```
mp_selector:    @ a,x,y = r0,r1,r2
    cmp r0,#0
    blt .else    @ NL on signed
    mov r0,r1
    bx lr
    .else:
    mov r0,r2
    bx lr
```

Qz. Q4 P2 5-bit ARM flags, (−15)−(−16): U: 17−16=1, no borrow, C=1; S: 1 ∈ [−16,15], V=0. Binary 0b00001 (N=0,Z=0). 16−18: U: needs borrow, C=0; S: −2 ∈ [−16,15], V=0. Binary 0b11110 (N=1,Z=0).

Qz. Q4 P3 LDR pre/post + scaled. Given mem 0x2000_0000..000F = 0x80..0x87,0x78..0x7F. task1 ([r0,#4]! twice): r2=0x8786_8584, r3=0x7B7A_7978. task2 ([r0],#8 then [r0,r2,ls1 #2] w/ r2=−1): first r3=0x8382_8180; after postinc r0=0x2000_0008; second r3 reads 0x2000_0008+(−4)=0x2000_0004 = 0x8786_8584. Store dest mem@0x200000030: 0x80,0x81,0x82,0x83,0x84,0x85,0x86,0x87. Prototypes: void task1(int *p1,int *p2); void task2(int *,int *,int); C for task1: *p2 = **p1; (*p2+1) = **p1+1;

21. Quiz 6a (flow ctrl)

Qz. Q6a P1 ldrd/strd + two ldrsb [r0,#4]!. ipt[i]=0x70+(i<<2) → bytes 0x70,0x74,...,0xB0. After ldrd r2,r3,[r0,#8]!: r2 = little-endian word at ipt+8 = 0x7C78_7470... wait, bytes are {0x80,0x84,0x88,0x8C}, so r2=0x8C_88_84_80. Then ldrsb pre-inc by 4 at 0xB0 → r2 = 0xFFFFFBB0 (sign-ext MSB=1).

Qz. Q6a P2 saturate to [−5,5]:

```
math_fun_s:    @ x→r0
    cmp r0,#-5
    blt .clamp_lo
    cmp r0,#5
    bgt .clamp_hi
    bx lr
    .clamp_lo: mov r0,#-5
    bx lr
    .clamp_hi: mov r0,#5
    bx lr
```

Qz. Q6a P3 sum of negative elts: int sum=0; for(i=0;i<N;i++){ if(arr[i]<0) sum+=arr[i];

```
mov r2,#0    @ sum
mov r3,#0    @ i
```

```
lp:
    cmp r3,r1
    bge done
    ldr r12,[r0,r3,ls1 #2]
    cmp r12,#0
    bge skip
    add r2,r2,r12
```

```
skip:
    add r3,#1
    b lp
done:
    mov r0,r2
    bx lr
```

Standard array-loop template: index, bound-cmp, scaled LDR, predicate, skip, increment, branch.

22. ASM Quick Ref

Immediate loads

```
ldrb r0,[r1]    @ *r1
ldrb r0,[r1,#4] @ *(r1+4)
ldrb r0,[r1,#4]! @ r1+=4; *r1
ldrb r0,[r1],#4 @ *r1; r1+=4
ldrb r0,[r1]    @ u8 zero-ext
```

```
ldrb r0,[r1]    @ s8 sign-ext
ldrh r0,[r1]    @ u16 zero-ext
ldrh r0,[r1]    @ s16 sign-ext
ldrd r0,r1,[r2,#8]! @ 64b
```

Reg-offset / stores

```
ldr r0,[r1,r2,ls1 #2] @ u32[]
ldrh r0,[r1,r2,ls1 #1]
ldrb r0,[r1,r2]
strb r0,[r1,#4]!
strh r0,[r1],#1
strh r0,[r1,#-2]
strd r0,r1,[r2],#8
```

MOV & pseudo-LDR (L20)

```
mov r0,r1
mov r0,#42    @ small imm
movw r0,#0xBEEF    @ low 16, zeros high
movt r0,#0xDEAD    @ high 16, preserve low
ldr r0,#0x12345678    @ pseudo
```

ldr Rd,=expr: 8-bit → mov.w; 16-bit → movw; 32-bit → PC-rel ldr + .word literal pool.

Stack

```
push {r4-r7,lr}    @ SP=20; low@low-addr
pop {r4-r7,pc}    @ restore + return
```

23. Directives & File Header

```
.section .text
.syntax unified
.thumb
.align 2
.global my_fn
.type my_fn, %function
my_fn:
    @ ...
    bx lr
.end
.word/.byte/.ascii → literals (TBB table, vector table). .weak name → placeholder. .thumb_set alias,target → symbol alias.
```

24. Registers & Exception

Low R0–R7 (some 16-bit Thumb limited); High R8–R12. R13=SP, R14=LR, R15=PC, R12=IP.

xPSR = {APSR, IPSR, EPSR} (overlapping views).

Exception auto-stack: HW pushes Handler {R0,R1,R2,R3,R12,LR,PC,PSR} before handler. need NOT re-save these.

25. Instructor Style

- Labels: <fn>_lp, _end, _then, _elseif, _else.
- Every ASM fn: (1) C-prototype comment, (2) In/Out/Other reg block, (3) .global/.type.
- GNU: lowercase insns; comments @.
- Return: bx lr (leaf) or pop {r4,pc} (preferred when pushed).
- Test harness: FUT=Function Under Test, CFN/AFN=C/ASM fn, UTF=Unit Test Fn, NUT=Number Under Test.
- ITTTT = 4-cond max.

26. Glossary

Term	Meaning
CCS	Condition Code Suffix (eq,ne,gt,...)
CEX	Conditional EXec (IT-based)
Op2	“Flexible 2nd operand”
CP range	Common Positive [0,2 ^N −1−1]
NUT	Number Under Test (flag analysis)
OUT	Operation Under Test
FUT	Function Under Test
Pseudo LDR	ldr Rd,=expr
Leaf/Stem	no BL / has BL
AAPCS	ARM Procedure Call Standard
EABI	Embedded App Binary Interface
MAC	Multiply-Accumulate
FD	Full-descending (stack)

27. C→ASM Checklist

For each C statement:

1. **Data type?** → LDR variant + offset multiplier.
2. **Inc order?** pre (l) vs post vs none.
3. **What's modified?** mem value (needs store-back), ptr, or both.
4. **Signed/unsigned?** → CCS (GT/LT vs HI/LO), shift (ASR vs LSR), load suf (SB/SH vs B/H).
5. **Single or branch?** single → Op2/CEX; multi → labeled branch.
6. **Leaf or stem?** any bl → push LR + callee-saved.
7. **Flags needed?** S-suffix if later branch relies on them.

28. Last-Minute Traps (Exam Gotchas)

- **Sign-ext on byte/half MSbit=1:** LDRSB 0x80→0xFFFFFFF80, LDRSH 0xB0A0→0xFFFFFB0A0. LDR-B/LDRH zero-ext.
- **Sign-ext at call site:** int8_t 0x81 as arg → R1=0xFFFFFFF81.
- **STRD / reg-offset:** neither exists. Don't write strd

- rL,rH,[Ra,Ri,LSL#3].
- **CBZ/CBNZ in IT:** illegal. Rn must be R0–R7. Forward only, ≤126B.
- **Cond B in IT:** legal ONLY as final slot.
- **#-imm in CMP:** cmp r0,#-5 silently becomes CMN r0,#5. Signed CCS still works.
- **Prefix ++ on *p:** MUST STR/STRB write-back the incremented mem value.
- **Ptr arith is scaled:** p32+= +4 bytes, not +1. (void*)p+= +1 byte.
- **Flag update requires S-suffix:** ADD ≠ ADDS. MUL never sets flags (only MULS sets N,Z).
- **UDIV/SDIV never set flags** (no S form). MUL has no C/V. **ARM sub flag:** C!=Borrow. Opposite of x86.
- **Signed vs unsigned divide:** ASR for int/2, LSR for uint/2. ASR on negative number OK; LSR gives garbage for negatives.
- **PUSH order:** always ascending reg# to ascending addr regardless of list syntax.
- **LDRD even-odd reg pair rule:** Cortex-M4 allows any pair; some cores require Rt even + Rt+1. Stick with consecutive regs.
- **BIC not &~:** C has no single “bit clear” operator; must spell as a & ~m. ASM is bic Rd,Rn,mask.
- **Op2 shift count:** MUST be immediate (Rn,LSL #n). Standalone LSL/LSR can take Rm for count.
- **GNU shifted-Op2:** dest register cannot be omitted — write add r0,r1,r1,ls1 #3, not add r0,r1,ls1 #3.
- **Q-format mul renorm:** Qm.n × Qm.n → Q2m.2n (in 2N-bit); shift right by n to renormalize.
- **V=signed, C=unsigned.** Picked via CCS. The HW sets both every arithmetic; you choose which to trust.
- **ADR range limited:** adr r1,table needs table within PC±4KB. Use ldr r1,=table if far.
- **TBB offset scale:** byte table stored as (case_label − table)/2 (Thumb 2-byte alignment).

29. Pattern Catalog: C → ASM

Integer arithmetic

C	ASM
c = a + b	add r0,r1,r2
c = a - b	sub r0,r1,r2
c = b - a	rsb r0,r1,r2 (Op2−Rn)
c = a * b	mul r0,r1,r2
c = a * b + d	mla r0,r1,r2,r3
c = d * a * b	mls r0,r1,r2,r3
c = a / b unsigned	udiv r0,r1,r2
c = a / b signed	sdiv r0,r1,r2
c = a % b unsigned	udiv rt,r1,r2 / mls r0,rt,r2,r1
c = -a	rsb r0,r1,#0
c = -a	mva r0,r1
(int64_t)a*b	adds rL,rL,#v / adc rH,rH,#0

Shifts / scaling

C	ASM
c = a << n	ls1 r0,r1,#n
c = (u)a >> n	lsr r0,r1,#n
c = (s)a >> n	asr r0,r1,#n
c = 2*a	ls1 r0,r1,#1
c = a/2 signed	asr r0,r1,#1
c = 3*a	add r0,r1,r1,ls1 #1
c = 5*a	add r0,r1,r1,ls1 #2
c = 7*a	rsb r0,r1,r1,ls1 #3
c = a - a/4 s	sub r0,r1,r1,asr #2
c = a + a/8 s	add r0,r1,r1,asr #3

Bit manipulation

C	ASM
c = a & b	and r0,r1,r2
c = a b	orr r0,r1,r2
c = a ^ b	eor r0,r1,r2
c = a & -b	bic r0,r1,r2
c = a -b	orn r0,r1,r2
c = (1<n)	orr r0,r0,#1,ls1 #n
c &= ~(1<n)	bic r0,r0,#1,ls1 #n
c ^= (1<n)	eor r0,r0,#1,ls1 #n
bool = c & (1<n)	tst r0,#1,ls1 #n
a &= ~(m<s); a = v<s	BIC mask + ORR value

Memory (pointers, arrays)

C	ASM
v = *p	ldr r0,r,[r1]
*p = v	str r0,[r1]
v = p[k]	ldr r0,[r1,#4k]
v = p[i]	ldr r0,[r1,r2,lsl #2]
v = *p+	ldr r0,[r1],#4
*p++ = v	str r0,[r1],#4
v = ++p	ldr r0,[r1,#4]!
v = *(int16_t*)p	ldrsh r0,[r1]
v = *(uint16_t*)p	ldrh r0,[r1]
v = *(int8_t*)p	ldrsh r0,[r1]
v = *(uint8_t*)p	ldrsh r0,[r1]
v = *(int64_t*)p	ldrdr r0,r1,[r2]
p = &arr[0]	ldr r0=arr or adr r0,arr

Flow control

C	ASM
if (a==b)	cmp / bne end
if (a==0)	cbnz r0,skip
while (*p)	ldrb + cbz r0,end
if (p==NULL) ret	cbz r1,end
return leaf	bx lr
return stem	pop {...,pc}
f(a,b)	mov r0,ra / mov r1,rb / bl f

30. Common Recipes

Signed saturate to [L, H]

```
cmp r0,#L
blt lo
cmp r0,#H
bgt hi
bx lr
lo: mov r0,#L / bx lr
hi: mov r0,#H / bx lr
```

Array: sum all elements (int)

```
mov r2,#0 @ sum
mov r3,#0 @ i
lp:
  cmp r3,r1 @ vs N
  bge end
  ldr r12,[r0,r3,lsl #2]
  add r2,r2,r12
  add r3,#1
  b lp
end: mov r0,r2 / bx lr
```

Array: conditional sum (negatives only)

```
mov r2,#0 / mov r3,#0
lp:
  cmp r3,r1
  bge end
  ldr r12,[r0,r3,lsl #2]
  cmp r12,#0
  bge skip
  add r2,r2,r12
skip:
  add r3,#1 / b lp
end: mov r0,r2 / bx lr
```

Array max (signed)

```
ldr r2,[r0] @ max=arr[0]
mov r3,#1
lp:
  cmp r3,r1
  bge end
  ldr r12,[r0,r3,lsl #2]
  cmp r12,r2
  ble skip
  mov r2,r12
skip:
  add r3,#1 / b lp
end: mov r0,r2 / bx lr
```

String length (strlen)

```
mov r1,r0
lp:
  ldrb r2,[r1],#1
  cbz r2,end
  b lp
end:
sub r0,r1,r0
sub r0,#1 @ don't count '\0'
bx lr
```

String copy (strcpy)

```
@ r0=dst, r1=src
lp:
  ldrb r2,[r1],#1
  strb r2,[r0],#1
  cbnz r2,lp
  bx lr
```

Memcopy (byte-wise)

```
@ r0=dst, r1=src, r2=n
mov r3,#0
lp:
  cmp r3,r2
  bge end
  ldrb r12,[r1,r3]
  strb r12,[r0,r3]
  add r3,#1
  b lp
end: bx lr
```

64-bit signed multiply+accumulate MAC

```
@ {rH:rL} += rA*rB (signed)
smial rL,rH,rA,rB
```

Q15 multiply (Q15 × Q15 → Q15)

```
@ r0,r1 are Q15 signed (16-bit in low)
smull r2,r3,r0,r1 @ Q30 in r3:r2
lsr r0,r2,#15 @ renorm Q30 -> Q15
@ add r0,r0,r3,lsl #17 if high bits matter
```

Unsigned modulo without UDIV-MLS

```
@ fallback: udiv + mul + sub
udiv r2,r0,r1
mul r2,r2,r1
sub r0,r0,r2
```

Pointer difference (elements)

```
@ (p - q) of typed pointers
sub r0,r0,r1 @ bytes
asr r0,r0,#shift @ /4 for int*, /2 for int16_t*
```

Assign bit-field (v to bits[s+w-1:s])

```
@ r0=target; v in r1; (mask=(1<<w)-1)
bic r0,r0,#mask,lsl #s
and r2,r1,#mask @ trim v
orr r0,r0,r2,lsl #s
```

Extract bit-field (bits[s+w-1:s] → Rd)

```
@ UBFX = unsigned bit field extract
ubfx r0,r1,#s,#w @ or manual lsr#s+and#mask
sbfx r0,r1,#s,#w @ signed (sign-extends)
```

31. HW Coverage Map

HW	Topic	Key instrs
HW5	Q-format encode/decode, Q15 MAC	shifts, mul
HW6	EABI args, 64-bit add/sub, modulo	ADDS/ADC, SUBS/ADC, SUBS
HW7	Shifts, Op2 scaling, bitwise	LSL/LSR/ASR, ORR, EOR
HW8	NZCV flags, CCS, branch	CMP/CMN, Bcc, SMLAL
HW9	LDR/STR addressing modes	basic/pre/post, reg offset
HW10	Pointers, LDRD, sign-ext	LDRSB/LDRSH, LDRD
HW11	If/if-else flow control (PL/NL/CEX)	IT blocks, cond B
HW12	Loops, null-term strings	CBZ, post-inc LDRB

32. Full HW11 P1 side-by-side

if (x>y) return (int32_t)*(++ptrA)*2; else return *(ptrB++)/2; R0=x, R1=y, R2=ptrA (int16_t*), R3=ptrB (int32_t*).

Part 1 (PL)

```
cmp r0,r1
bgt pl_then
pl_else:
  ldr r0,[r3],#4
  asr r0,r0,#1
  b pl_end
pl_then:
  ldrsh r0,[r2,#2]!
  lsl r0,r0,#1
  bx lr
pl_end:
```

Part 2 (NL)

```
cmp r0,r1
ble nl_else
nl_then:
  ldrsh r0,[r2,#2]!
  lsl r0,r0,#1
  b nl_end
nl_else:
  ldr r0,[r3],#4
  asr r0,r0,#1
  bx lr
nl_end:
```

Part 3 (CEX, ITTEE le)

```
cmp r0,r1
ittee le
ldrlt r0,[r3],#4
asrle r0,r0,#1
ldrshgt r0,[r2,#2]!
lslgt r0,r0,#1
bx lr
```

Takeaways: choose CCS suffix=NL-cond in IT header; T-slots match, E-slots inverse. Pre/post-idx LDRs fully legal in IT.

33. Full HW11 P2 piecewise f(x)

f(x)={-10 if x<-5; +10 if x>=5; 2x else}

C

```
int f(int x){
  int y;
  if (x<-5) y=-10;
  else if (x>=5) y=10;
  else y=2*x;
  return y;
}
```

CMP chain (NL at each arm)

```
cmp r0,#-5 @ = CMN r0,#5
bge elseif
ldr r0,-=10
b end
elseif:
  cmp r0,#5
  blt else_blk
  b end
else_blk:
  lsl r0,r0,#1 @ 2*x
end:
bx lr
```

CEX (chained IT + cond B-as-last-slot)

```
cmp r0,#-5
itt lt
ldrlt r1,-=10 @ T1
blt end @ T2 (last slot!)
cmp r0,#5
ite ge
ldrge r1,=10
lsltt r1,r0,#1 @ uses orig x in R0
end:
mov r0,r1
bx lr
```

Takeaway: stage result in R1 so R0 (holding x) survives until 2nd CMP. Cond B in IT is legal only as last slot.

34. Full HW11 P3 compound OR

if (x<=20 || x>=25) ... else ... R0=x, R1=ptrA (uint16_t*), R2=ptrB (uint32_t*). Unsigned /4 ⇒ LSR.

CMP short-circuit

```
cmp r0,#20
ble then
bge else
ldr r0,[r1],#2!
lsl r0,r0,#2
b else
else:
  ldr r0,[r2,#4]!
  lsr r0,r0,#2
end:
bx lr
```

CEX (1st test plain B, 2nd test ITTEE)

```
cmp r0,#20
bgt check25
ldrh r0,[r1,#2]!
lsl r0,r0,#2
bx lr
check25:
  cmp r0,#25
  ittee ge
  ldrhge r0,[r1,#2]!
  lslge r0,r0,#2
  ldrlt r0,[r2,#4]!
  larlt r0,r0,#2
  bx lr
```

Takeaway: 1st of two OR sub-tests can't pack into IT (cond B only at end); handle with plain bgt, then ITTEE the tail.

35. Full HW12 P2 string copy (upper-case)

C (while with priming read)

```
void mp_str_cpy_2_caps_while_c(char *d, char *s){
  char ch = *s++;
  while (ch){
    ch -= 0x20;
    *d++ = ch;
    ch = *s++; @ duplicate read
  }
  *d++ = ch; @ writes '\0'
}
```

Part 1 ASM

```
mp_str_cpy_2_caps_while_s:
  ldrb r2,[r1],#1 @ priming
lp:
  cbz r2,end
  sub r2,r2,#0x20
  strb r2,[r0],#1
  ldrb r2,[r1],#1 @ in-loop DUP
  b lp
end:
```

```
strb r2,[r0],#1 @ '\0' (r2=0 via CBZ)
bx lr
```

Part 2 C (while(1)+break)

```
void mp_str_cpy_2_caps_while_brk_c(char *d, char *s){
  while (1){
    char ch = *s++;
    if (ch == 0) { *d++ = ch; break; }
    ch -= 0x20;
    *d++ = ch;
  }
}
```

Part 3 ASM (single LDRB text)

```
mp_str_cpy_2_caps_while_brk_s:
lp:
  ldrb r2,[r1],#1
  cbz r2,end
  sub r2,r2,#0x20
  strb r2,[r0],#1
  b lp
end:
  strb r2,[r0],#1
  bx lr
```

Takeaway: priming-read style → two LDRB texts (easy error source), while(1)+break → one LDRB. Same number of dynamic LDRBs (N+1).

36. Test Framework / Naming

- FUT = Function Under Test.
- CFN = C Function impl. AFN = ASM Function impl.
- UTF = Unit Test Function.
- NUT = Number Under Test (flag-bit analysis).
- OUT = Operation Under Test.
- CP = Common Positive range $[0, 2^{N-1}-1]$ where signed/unsigned agree.
- Typical name: mp_<op>_<type>_<c|s> e.g. mp_add_64bit_s.
- Each fn has: (1) /* C-prototype comment */, (2) In/Out/Other reg block, (3) .global + .type %function.

37. GPIO Quick Snippets (L6-7)

These appear on earlier labs; unlikely on the final but useful if asked:

Direct ODR access

```
@ set PA5 high via ODR OR (read-modify-write)
ldr r0,=GPIOA_ODR
ldr r1,[r0]
orr r1,r1,#(1<<5)
str r1,[r0]
```

Atomic BSRR

```
@ set PA5: write 1<<5 to BSRR[15:0]
ldr r0,=GPIOA_BSRR
mov r1,#(1<<5)
str r1,[r0]
```

```
@ reset PA5: write 1<<(5+16) to BSRR[31:16]
mov r1,#(1<<(5+16))
str r1,[r0]
```

BSRR advantage: single atomic write, no read-modify-write race with ISR modifying other bits of same port.

38. Exception Handler Entry (L8)

HW auto-stacks {R0, R1, R2, R3, R12, LR, PC, PSR} on current stack. Handler inherits LR=EXC_RETURN magic value (0xFFFFFFFx) — BX LR returns from the exception.

- Handler need NOT re-save R0–R3, R12, LR, PC, PSR.
- Handler DOES need to save R4–R11 if it uses them.
- EXC_RETURN picks between MSP/PSP and thread/handler mode.

39. Memory Regions (F412 / G431)

Region	Start	Use
Flash	0x08000000	code, .rodata, literal pool
SRAM	0x20000000	.data, .bss, stack
Peripheral	0x40000000	GPIO/UART/TIM regs
System	0xE0000000	NVIC, SysTick

at 0x08000000 by default: SP init (word 0), reset handler (word 1), then NMI, HardFault, ... IRQs.

40. Endianness Drills

ARM Cortex-M4 = little-endian. Byte at lowest addr = LSB of the multi-byte word.

Bytes @addr	32-bit word
0x01,0x02,0x03,0x04	0x04030201
0x80,0x00,0x00,0x00	0x00000080
0xFF,0xFE,0xFD,0xFC	0xFCFDFEFF

LDR alignment: word LDR needs 4-byte aligned addr; LDRH needs 2-byte aligned; LDRB any. Unaligned LDR may fault (Cortex-M4 allows, but slow).

41. Numeric Ranges

Type	Min	Max
uint8_t	0	255
int8_t	-128	127
uint16_t	0	65535
int16_t	-32768	32767
uint32_t	0	2 ³² -1
int32_t	-2 ³¹	2 ³¹ -1
int64_t	-2 ⁶³	2 ⁶³ -1

TC encode: neg → add 2^N (wrap). **TC decode:** if MSB=1, subtract 2^N.

Hex	TC dec	sign bit
0x00	0	+
0x7F	127	+(max+)
0x80	-128	-(min-)
0xFF	-1	-

42. Full ASM for HW10 P1 (4 lines)

```
uint8_t *p8a = 0x20000100; uint8_t *p8b = 0x20000100;
Lines:
@ Line 4: *p8b++ = *(p8a + 2);
ldrb r2,[r0,#2]
strb r2,[r1],#1

@ Line 5: *p8b++ = ++*(p8a + 4);
ldrb r2,[r0,#4]
add r2,r2,#1
strb r2,[r0,#4] @ write-back!
strb r2,[r1],#1

@ Line 6: *p8b++ = ++*p8a++;
ldrb r2,[r0],#1
add r2,r2,#1
strb r2,[r0,#-1] @ back to old addr
strb r2,[r1],#1

@ Line 7: *p8b++ = *p8a++;
ldrb r2,[r0],#1
strb r2,[r1],#1
```

Prefix ++ on a *-deref ⇒ STR write-back after ADD. Postfix on pointer ⇒ post-index.

43. Full HW9 P1 ASM

16 bytes 0x00..0xFF @ 0x20000000. Little-endian word layout:

Addr	Word
0x20000000	0x03020100
0x20000004	0x07060504
0x20000008	0x0B0A0908
0x2000000C	0x0F0E0D0C

```
ldr r4=0x20000000 @ r4 = 0x20000000
ldr r0,[r4,#4] @ r0 = 0x07060504
@ r4 unchanged
ldr r1,[r4,#2]! @ r4 = 0x20000002
@ r1 = 0x05040302
ldr r2,[r4],#4 @ r2 = 0x05040302
@ r4 = 0x20000006
ldr r3,[r4] @ r3 = 0x09080706
```

44. Common Exam Traps — Extended

- **Misreading LDR alignment:** ldr r0,[r4,#2]! loads FOUR bytes starting at r4 + 2 (so bytes 2,5), NOT two bytes.
- **LDRD register pair is consecutive:** ldrd r2,r3,... — r3 must be r2+1. ldrd r2,r5,... is illegal.
- **STRB silently discards upper 24 bits.** Writing 0x12345678 via STRB stores only 0x78.
- **ASR vs LSR on boundary values:** asr r0,r1,#1 where r1=0x80000000 gives 0xC0000000 (sign fill); lsr gives 0x40000000.
- **CMP immediate range:** 12-bit modified immediate. Big constants must be loaded via ldr r12,*big, then CMP reg-to-reg.
- **MOVs vs MOV:** MOVs sets flags. movs r0,#0 clears Z=1, N=0. Handy for zeroing AND testing.
- **UDIV/SDIV by zero:** returns 0 on Cortex-M4 (no exception by default). Check divisor first if needed.
- **Integer overflow on MUL:** 32-bit MUL truncates; bits above 31 silently lost. Use SMULL/UMULL to preserve.
- **EABI stack alignment:** before a BL, SP must be 8-byte aligned. Push odd # of regs + align dummy, or push pairs.
- **CBZ target must be AHEAD in memory.** Backward CBZ illegal.
- **IT block terminates early** if interrupted by LDR literal pool or label.
- **Cortex-M4 does NOT support ARM mode** — Thumb-2 only. Forgetting .thumb directive breaks everything.
- **Tail-call optimization:** b func (not bl) as the last instruction of a leaf-after-BL chain re-uses caller's LR — saves a pop/-push pair.

- **Self-modifying stack via PUSH+POP order:** PUSH preserves list regardless of order; POP restores in the same layout. Don't rely on list-syntax ordering.

45. Worked Flag-Compute Drill

Example: 8-bit sys ($C_N = 256$), compute flags for r0 + r1 where r0=0xA3 (163), r1=0x7F (127).
Unsigned: 163 + 127 = 290. 290 > U_{max} = 255 ⇒ 290 - 256 = 34. C=1.
Signed: r0 = 0xA3 has MSB=1 so value = 163 - 256 = -93; r1 = 0x7F = +127. -93 + 127 = 34 ∈ [-128, 127] ⇒ V=0.
Result binary: 34 = 0b00100010. N=0 (MSB). Z=0.
Final flags: N=0, Z=0, C=1, V=0.

Example 2: 8-bit r0 = 0x7F (+127) + r1 = 0x01. Unsigned: 128 < 256, no carry, C=0. Signed: 127 + 1 = 128 > 127 ⇒ V=1 (corrected to -128). Binary 0b10000000. N=1, Z=0.
Subtraction drill: r0 - r1 where r0 = 0x20 (32), r1 = 0x30 (48). Unsigned 32 - 48 needs borrow ⇒ C=0. 32 - 48 = -16 ∈ [-128, 127] ⇒ V=0. Result -16 = 0xF0. N=1, Z=0.

46. Thumb-2 Reserved / Special Names

Name	Alias
R13	SP (stack ptr)
R14	LR (link reg)
R15	PC (program counter)
R12	IP (intra-proc scratch)

Don't use as general regs: SP (push/pop only), PC (branch only). LR is free-ish but must preserve if stem.
“**Low**” registers (R0–R7): available to most 16-bit Thumb insns. Some insns (e.g. CBZ) restrict to these.

47. Two's Complement Speed Sheet

Negate (2's comp): invert all bits, add 1.

8-bit hex	dec	binary
0x01	+1	0000 0001
0x7F	+127	0111 1111
0x80	-128	1000 0000
0xFF	-1	1111 1111
0xC0	-64	1100 0000
0xA3	-93	1010 0011

Quick rule: 8-bit TC dec = (unsigned dec) - 256 when MSB=1.
Sign-ext byte to 32-bit: if top bit of 8-bit value is 1, set high 24 bits to 1s: 0x81 → 0xFFFFF81. Otherwise 0 fill: 0x7E → 0x0000007E.

48. Stack Drawings

Initial state — SP = 0x20001020:

0x20001020 (top, nothing pushed yet)

After push {r4,r8} (2 regs, 8B):

0x20001018	R4	<-- SP
0x2000101C	R8	
0x20001020	(was top)	

Regs stored at LOW addr to HIGH addr in **ascending reg order**: R4 low, R8 high.

After pop {r4,r8}: SP restored to 0x20001020; slots unchanged in memory (not erased, just forgotten).

49. Condition-Flag Quick Lookup

After CMP r0, r1 (i.e. r0 - r1):	
If ...	then
r0 == r1	Z=1, C=1, N=0, V=0
r0 > r1 (u)	Z=0, C=1
r0 < r1 (u)	C=0
r0 > r1 (s)	Z=0, V==N
r0 < r1 (s)	V ≠ N
r0 < 0, r1 ≠ 0 (s)	N depends on r1 sign

Branch chooser: if C gives ==/!=, just use EQ/NE — works for both signed and unsigned.

50. Calling-Convention Templates

Leaf fn (no BL, no R4+ used)

```
leaf_fn:
@ use R0-R3, R12 freely
@ compute result in R0
bx lr
```

Leaf fn but uses R4 for loop

```
leaf_with_r4:
push {r4} @ save callee-saved
@ ...
pop {r4}
bx lr
```

Stem fn (calls another fn)

```
stem_fn:
push {r4-r7,lr} @ save callee + LR
mov r4,r0 @ stash caller's args
mov r5,r1
@ ... prep R0-R3 for call ...
bl inner_fn
@ ... use result ...
pop {r4-r7,pc} @ restore + return
```

Stem calling in loop

```
stem_loop:
push {r4-r7,lr}
mov r4,r0 @ base ptr
mov r5,r1 @ N
mov r6,#0 @ i
```

```
lp:
cmp r6,r5
bge end
ldr r0,[r4,r6,ls! #2]
bl process
@ R0 = result; store it
str r0,[r4,r6,ls! #2]
add r6,#1
b lp
end:
pop {r4-r7,pc}
```

51. Operator Precedence Cheat (C)

- Highest → lowest:
1. $() \square \rightarrow .$ postfix ++/-
 2. prefix ++/- + - ! * (deref) & (type) sizeof
 3. * / %
 4. + -
 5. << >>
 6. < <= > >=
 7. == !=
 8. & ^ | (bitwise)
 9. && ||
 10. ? :
 11. = += -= ...
 12. ,

Right-to-left for L2 (prefix ops), L11 (assignment), L12 (ternary). Rest LTR.

Common trap: *r++ = (*r++) not (*r)++ (postfix has higher precedence than deref). Use (*r)++ if you mean value-increment.

52. HW5 Walkthrough (all 5 problems)

P1 (8pts) Encode 2.75 in UQ3.5

$N = 8, n = 5$, scale = 2⁵ = 32. $I = 2.75 \cdot 32 = 88 = \mathbf{0x58}$.

P2 (16pts) Decode Q3.4 codes

- (a) 0x77 = 119, MSB=0 (positive), $f = 119/16 = \mathbf{+7.4375}$.
(b) 0xAA = 170, MSB=1 (negative), $I = 170 - 256 = -86$, $f = -86/16 = \mathbf{-5.375}$.

P3 (10pts) Decode Q2.5 code 0b10011001

$U = 153$, MSB=1, $I = 153 - 256 = -103$, $f = -103/32 = \mathbf{-3.21875}$.

P4 (32pts) Encode in Q3.4

- (a) $0.5 \cdot 16 = 8 \rightarrow \mathbf{0x08}$
(b) $6.73 \cdot 16 = 107.68 \rightarrow 108 \rightarrow \mathbf{0x6C}$ (err 0.02)
(c) $-2.25 \cdot 16 = -36$, +256 = 220 → **0xDC**
(d) $-4.5 \cdot 16 = -72$, +256 = 184 → **0xB8**

P5 (44pts) Q15 MAC: $f_1 f_2 + f_3 f_4$

	f	$\times 2^{15}$	hex	0.5
Encode (multiply by 2 ¹⁵):	0.25	8192	0x2000	
	0.125	4096	0x1000	
	0.0625	2048	0x0800	

Multiply Q15 (Q15 × Q15 = Q30; renorm by ≫ 15):
 $0.5 \cdot 0.25 = 0x4000 \cdot 0x2000 = 0x08000000 \gg 15 = \mathbf{0x1000}$ (= 0.125)
 $0.125 \cdot 0.0625 = 0x1000 \cdot 0x0800 = 0x00800000 \gg 15 = \mathbf{0x0100}$ (= 0.0078125)
Add: $0x1000 + 0x0100 = \mathbf{0x1100} = 4352/32768 \approx \mathbf{0.1328125}$.

53. HW6 Walkthrough (7 problems)

P1 PUSH on full-descending stack

SP=0x20001020; push {r7,r5,r8,r6}; Ri=(1<i<1)+(1<i<2i).
P1.1 SP after = 0x20001020-16 = **0x20001010**.
P1.2 R5 = 32 + 1024 = **0x00000420**.
P1.3 Storage low→high: R5@..1010, R6@..1014, R7@..1018, R8@..101C. Word@..1014 = R6 = 64 + 4096 = **0x00001040**.
P1.4 Byte@0x20001014 = LSB of R6 = **0x40** (LE).

P2 EABI unsigned promote

Call fn1((uint8_t)0xF, (uint8_t)0xFF, 0xFF, 0xFFFF); fn1 takes uint32_t.
R0=0x0000000F, R1=0x000000FF, R2=0x00000FFF,
R3=0x0000FFFF (zero-ext).

P3 EABI signed promote

Call fn2((int8_t)0xF, (int8_t)0xFF); fn2 takes int32_t.
R0 = 0x0000000F (MSB=0).
R1: 0xFF = -1 as int8; sign-ext → **0xFFFFFFFF**.

P4 Q15.16 return

Encode 3.25: $I = 3.25 \cdot 2^{16} = 3 \cdot 0x10000 + 0x4000 = \mathbf{0x00034000}$.
32-bit return → R0.

P5 64-bit ADD

Convention: R0=lo(A), R1=hi(A), R2=lo(B), R3=hi(B).

```
mp_add_64bit_s:
add r0,r2 @ lo+lo, sets C
adc r1,r3 @ hi+hi+C
bx lr
```

T1: 0x88000000+0x89000000 = 0x11100000 ⇒ lo=0x11000000, C=1; hi=3+2+1=6.
T2: 0x78000000+0x79000000 = 0xF1000000 ⇒ lo=0xF1000000, C=0; hi=3+2+0=5.

P6 64-bit SUB

```
mp_sub_64bit_s:
subs r0,r2 @ lo-lo, C=borrow
sbc r1,r3 @ hi-hi-(1-C)
bx lr
```

T1: 0x70000000-0x80000000 needs borrow ⇒ lo=0xF0000000, C=0; hi=0x38-0x02-1=0x35.
T2: 0x80000000-0x80000000=0, no borrow ⇒ C=1; hi=0x38-0x28-0=0x10.

P7 UDIV+MUL modulo

$A \% B = A - (A/B) \cdot B$.

mp_umod_32bit_s:

```
udiv r2,r0,r1
mul r2,r2,r1
sub r0,r0,r2
bx lr
```

A=17, B=5: 17/5 = 3, 3 · 5 = 15, 17 - 15 = 2.

Alt (1 fewer instr): udiv r2,r0,r1 / mls r0,r2,r1,r0 (mls Rd,Rn,Rm,Ra → Rd=Ra-Rn·Rm).

54. HW7 Walkthrough (6 problems)

P1 Q15.16 from SMULL

After smull r1,r0,r0,r1 (R0:HW, R1:LW), extract bits b47..b16 to put result in R0:

```
smull r1,r0,r0,r1
ls! r0,#16 @ HW<<(target_MSB-31)
add r0,r0,r1,lsr #16
```

Pattern: ls! HW by (target MSB - 31); lsr LW by n. Q31: ls!1+lsr31. Q15.16: ls!16+lsr16. Q16.15: ls!17+lsr15.

P2 LSLS/LSRS and C

R0=0xDD = 0b11011101.
ls!s r1,r0,#1 → R1=0x1BA, C = b7 of R0 = 1.
lsrs r2,r0,#3 → R2=0x1B, C = b2 of R0 = 1.

P3 One-instr scaling (signed)

A	ASM
16B	ls! r0,r1,#4
17B	add r0,r1,r1,ls! #4
15B	rsb r0,r1,r1,ls! #4
B/16	asr r0,r1,#4
17B/16	add r0,r1,r1,asr #4
15B/16	sub r0,r1,r1,asr #4

P4 BIC+ORR replace nibble

$0xH_3H_2H_1H_0 \rightarrow 0xH_3H_2AH_0$ (set bits[7:4]=0xA):

```
ldr r1,#15
bic r0,r0,r1,ls! #4
ldr r2,r0
orr r0,r0,r2,ls! #4
```

P5 Hand-eval BIC/ORR/EOR

R0=0xAD, R1=0x03:
bic r3,r0,r1 = 0xAD & ~0x03 = 0xAD & 0xFC = **0xAC**
orr r4,r3,r1 = 0xAC | 0x03 = **0xAF**
eor r5,r0,r1 = 0xAD ^ 0x03 = **0xAE**

P6 Insert digit at end

(b) Shift left + insert 3 at bottom:

```
ls! r0,#4
orr r0,#3
```

(c) Shift right + insert 7 at top (16-bit val):

```
lsr r0,#4
ldr r1,#7
ls! r1,#12
orr r0,r1
```

55. Worked CCS Drill

Setup: 6-bit system $C_N = 64$. After cmp r0,r1 where r0 = 22, r1 = 20 (or equivalently r0 = -20, r1 = -22):
Compute r0 - r1 = 2 in 6-bit:
• **Unsigned:** no borrow ⇒ C=1.

- **Signed:** $2 \in [-32, 31]$, no overflow $\Rightarrow V=0$.
- $N=0$, $Z=0$.

Branch evaluation (all CCS):

CCS Logic	Value	Branch?
EQ $Z=1$	0	no
NE $Z=0$	1	yes
GT $V=N$ and $Z=0$	1	yes
GE $V=N$	1	yes
LT $V \neq N$	0	no
LE $V \neq N$ or $Z=1$	0	no
HI $C=1$ and $Z=0$	1	yes
HS $C=1$	1	yes
LO $C=0$	0	no
LS $C=0$ or $Z=1$	0	no

Same flags from a different CMP: $-20 - (-22)$: in unsigned-mapped form $44 - 42 = 2$. Same flags. CCS picks same way.

56. Full HW9 P3 Tasks

Task 1 (basic offsets)

```
ldr r2,[r0,#4]      @ r2 = ipt+4 word = 0x70605040
ldr r3,[r1,#0]       @ store via opt
@ printout: 0x70605040
ldr r2,[r0,#0xC]
@ printout: 0xF0E0D0C0
```

After: R0 unchanged, R1 unchanged.

Task 2 (pre-index #4!)

```
ldr r2,[r0,#4]!      @ r0+=4 first
                    @ r0=0x...14, r2=0x70605040
ldr r2,[r0,#4]!      @ r0=0x...18, r2=0x80A09080
```

After: R0=0x20001018.

Task 3 (post-index #4)

```
ldr r2,[r0],#4      @ r2=0x30201000, then r0+=4
ldr r2,[r0],#4      @ r2=0x70605040, then r0+=4
```

After: R0=0x20001018.

Task 4 (register-offset, r2=1)

```
ldr r3,[r0,r2,ls1 #2] @ r2=1: addr=ipt+4
                    @ r3=0x70605040
add r2,#1
ldr r3,[r0,r2,ls1 #2] @ r2=2: addr=ipt+8
                    @ r3=0x80A09080
add r2,#1              @ r2=3
```

After: R0 unchanged, R2=3, R3=0xB0A09080.

57. Bonus: Bit-Twiddle Idioms

Idiom	ASM
test bit n	tst r0,#1,ls1 #n
isolate low n bits	ubfx r0,r1,#0,#n
isolate top n bits	lsr r0,r1,#(32-n)
clear low n bits	bfc r0,#0,#n
swap nibbles in byte	ror r0,r1,#4 (limited)
swap halves of word	ror r0,r1,#16
abs (signed)	cmp r0,#0/1t lt/rsblt r0,#0
sign of signed n	asr r0,r1,#31 (-1 or 0)
count zero bits up to LSB	rbit/clz (rev+CLZ)

58. Common ASM-File Skeleton

```
@ =====
@ File: mp_my_func.s
@ Purpose: <brief>
@ Author: <name>
@ Prototype: int mp_my_func(int x, int y);
@ =====

.section .text
.syntax unified
.thumb
.align 2

@ -----
@ In:  R0 = x (int32_t)
@     R1 = y (int32_t)
@ Out: R0 = result
@ Other: R12 scratch
@ -----

.global mp_my_func_s
.type mp_my_func_s, %function
mp_my_func_s:
    push {lr}
    @ ... body ...
    pop {pc}
.end
```

59. Number-Format Speed Tables

8-bit signed (TC) reference

Hex	U	S	Hex	U / S
0x00	0	0	0x80	128 / -128
0x01	1	+1	0x81	129 / -127
0x10	16	+16	0xC0	192 / -64
0x40	64	+64	0xE0	224 / -32
0x7F	127	+127	0xFF	255 / -1

16-bit signed boundaries

Hex	U	S
0x0000	0	0
0x7FFF	32767	+32767
0x8000	32768	-32768
0xFFFF	65535	-1

Q15 example values

$0.5 = 0x4000$, $0.25 = 0x2000$, $0.125 = 0x1000$, $0.0625 = 0x0800$, $-0.5 = 0xC000$, $-1.0 = 0x8000$ (saturates), $+0.99997 = 0x7FFF$ (max).

60. Quick Bcc Decoder

Bcc	“Branch if”	Flags
B	always	—
BEQ	==	Z=1
BNE	!=	Z=0
BHS/BCS	$u \geq$	C=1
BLO/BCC	$u <$	C=0
BMI	N=1 (negative result)	N=1
BPL	N=0 (zero or pos)	N=0
BVS	V=1 (signed overflow)	V=1
BVC	V=0	V=0
BHI	$u >$	C=1 and Z=0
BLS	$u \leq$	C=0 or Z=1
BGE	$s \geq$	V=N
BLT	$s <$	$V \neq N$
BGT	$s >$	$V=N$ and $Z=0$
BLE	$s \leq$	$V \neq N$ or $Z=1$

61. HW8 Walkthrough (NZCV+CCS)

P1 (42pts) Compute NZCV by hand (5-bit)

$C_N = 32$, $U_{\max} = 31$, $S_{\min} = -16$, $S_{\max} = 15$.

Procedure per add/sub:

1. Convert negatives to unsigned via $+C_N$.
2. Compute as if unsigned in $[0, C_N]$. If result wraps $\Rightarrow C$ (add: $C=1$; sub: borrow $C=0$).
3. Treat the SAME binary as signed. If result $\notin [S_{\min}, S_{\max}]$, $V=1$, correct by $\pm C_N$.
4. $N=MSB$ of binary; $Z=(value=0)$.

Drill 1: $(-15) + (-5)$.

U: $17 + 27 = 44 \rightarrow 44 - 32 = 12$, **C=1**.

S: $-15 + (-5) = -20 < -16$, $+32 = 12$, **V=1**.

Binary 0b011001, $N=0$, $Z=0$.

Drill 2: $11 - 18$.

U: $11 - 18 = -7 \rightarrow +32 = 25$, borrow $\Rightarrow C=0$.

S: $11 - 18 = -7 \in [-16, 15]$, **V=0**.

Binary 0b110011, $N=1$, $Z=0$.

Drill 3: $7 + 9$ (5-bit).

U: $7 + 9 = 16 \in [0, 31]$, no carry, **C=0**.

S: $7 + 9 = 16 > 15$, **V=1**, $-32 = -16$.

Binary 0b10000, $N=1$, $Z=0$. (Signed reads as -16 , unsigned reads as 16.)

P2 CCS table (after CMP)

Compute flags from $\text{cmp } r0,r1$ as $r0 - r1$, then evaluate every CCS. Equal-magnitude pairs (e.g. $22 - 20$ vs $-20 - (-22)$) give identical flags because $(-20) + C_N - ((-22) + C_N) = 22 - 20 = 2$.

Trick: CCS is **flag-driven**, not value-driven. Two CMPs that produce identical flags branch identically.

P3 max() via NL branch

```
int max(int a,int b){ return a>b ? a : b; }
```

```
max:
    cmp r0,r1
    blt .else      @ NL: branch on !(a>b)
    bx lr          @ then: r0 already = a
    .else:
    mov r0,r1      @ r0 = b
    bx lr
```

P4 64-bit sum-of-squares with SMLAL

$\text{int64_t sum}=0$; for($i=1$; $i \leq n$; $i++$) $\text{sum} += i*i$;

```
ldr r2,0      @ sum_lo
ldr r3,0      @ sum_hi
.lp:
    cmp r0,r1
    bgt .end
    smlal r2,r3,r0,r0 @ {hi:lo}+=i*i
    add r0,#1
    b .lp
```

```
.end:
    mov r0,r2      @ lo -> R0
    mov r1,r3      @ hi -> R1
    bx lr
```

Key: SMLAL R10,Rhi,Rn,Rm: signed $32 \times 32 \rightarrow 64$ -bit accumulate. Pair with EABI 64-bit return (R0=lo, R1=hi).

62. Worked Inc/Dec Drills

Init: $\text{uint8_t arr}[5,7,9,11]$; at $0x20000100$. $\text{uint8_t *p} = \text{arr}$;

```
x = *p++;
ldrb r0,[r1],#1
After: x=5, p=0x101.
```

```
x = *p++;
ldrb r0,[r1,#1]!
After: x=7, p=0x101.
```

x = (*p)++; (return old, increment mem)

```
ldrb r0,[r1]      @ x = old
strb r2,r0,#1      @ writeback
After: x=5, mem[0x100]=6, p unchanged.
```

x = ++(*p); (increment mem, return new)

```
ldrb r0,[r1]
add r0,r0,#1
strb r0,[r1]
After: x=6, mem[0x100]=6, p unchanged.
```

x = *p++;
“post-inc p, but modify mem at OLD addr, return new value of mem”

```
ldrb r0,[r1],#1 @ x=*old_p, p++
add r0,r0,#1
strb r0,[r1,-1] @ writeback to OLD addr
After: x=6, mem[0x100]=6, p=0x101.
```

```
*q++ = *p++;
ldrb r0,[r1],#1
strb r0,[r2],#1
```

63. Conditional Pre/Post LDR Inside IT

Both LDR{c} and STR{c} support pre/post-index inside IT blocks. Pattern from HW11 P1 P3:

```
cmp r0,r1
ittee le
ldrlie r0,[r3],#4 @ post-idx, T
asrlie r0,r0,#1
ldrshgt r0,[r2,#2]! @ pre-idx, E
lsllgt r0,r0,#1
bx lr
```

Suffix order: <insn><suffix><c> for shifts; <insn><c> for plain. So $\text{ldrshgt} = \text{LDRSH}$ on GT.

64. Loops — Detailed Count

For an N-element loop with:

- N iters $\Rightarrow$ N body executions, N+1 cmp+branch checks.
- Top-tested while: priming-read style $\Rightarrow$ N+1 LDRBs (1 priming + N in loop).
- while(1)+break $\Rightarrow$ N+1 LDRBs (one per iter incl. exit iter).

Same dynamic LDRB count between the two; the break-style just removes one source-line of code.

do-while (always ≥ 1 iter)

```
.lp:
    @ body always runs first
    cmp r0,r1
    blt .lp
```

Counted loop with downto

```
mov r4,r5      @ count = N
.lp:
    cbz r4,.end
    @ body
    sub r4,r4,#1
    b .lp
.end:
```

Trade: downto uses CBZ (no flags churn) and single SUBS. Up-counting needs CMP each iter.

65. Switch Variants

Naive CMP/BEQ ladder

```
cmp r0,#65      @ 'A'
beq case_a
cmp r0,#66
beq case_b
cmp r0,#67
beq case_c
b default
```

Stacked-label fall-through

```
case_a:          @ no insn here = falls through
case_b:
case_c:
    ldr r0,#80      @ same body for A, B, C
    b .end
```

TBB jump table (O(1))

```
cmp r0,#3
bge default
adr r1,tbl
tbb [r1,r0]      @ PC = PC+4+2*tbl[r0]
case_0:
    @ ...
    b .end
case_1:
    @ ...
    b .end
case_2:
    @ ...
    b .end
.align 1
tbl:
    .byte (case_0-tbl)/2
    .byte (case_1-tbl)/2
    .byte (case_2-tbl)/2
```

TBH (16-bit table, longer jumps)

```
tbb [r1,r0,LSL #1] @ table is .hword
```

66. C-ASM Pitfall Quiz

Q: $\text{int8_t } b = 0x80$; $\text{int32_t } x = b$; — value of x?

A: **0xFFFFFFFF80** (sign-extended).

Q: $\text{uint8_t } b = 0x80$; $\text{int32_t } x = b$; — value of x?

A: **0x00000080** (zero-extended; the unsigned bool char gets zero-padded).

Q: After $\text{ldr } r0,[r1,\#2]!$ with $r_1 = 0x100$, what's r_1 after?

A: $r_1 = 0x102$ (pre-index updates BEFORE).

Q: After $\text{ldr } r0,[r1,\#2]$ with $r_1 = 0x100$, what address was loaded from?

A: **0x100** (post-index updates AFTER).

Q: push {r5,r4,r6} on SP=0x1020. Where is r6?

A: 0x101C (highest addr regs stored ascending by reg#).

Q: $\text{p32} + 1$ where p32 is $\text{int32_t } * = 0x100$. Result?

A: 0x104 (pointer arith scaled by sizeof(int32_t)).

Q: adds r0,r1,r2 where $r1=0xFFFFFFFF$, $r2=0x1$. Result and flags?

A: $r0=0x00000000$, $Z=1$, $C=1$, $N=0$, $V=0$.

Q: CBZ inside IT block — legal?

A: **No**. CBZ/CBNZ forbidden in IT.

Q: $\text{asr } r0,r0,\#1$ on $r0=0x80000000$. Result?

A: 0xC0000000 (signed shift, sign bit replicated).

Q: $\text{lsr } r0,r0,\#1$ on $r0=0x80000000$. Result?

A: 0x40000000 (logical shift, zero fill).

Q: $f((\text{int8_t})-1)$ where f takes int32_t . Value in R0?

A: 0xFFFFFFFF.

Q: $\text{strb } r0,[r1]$ where $r0=0xDEADBEEF$. Memory written?

A: 0xEF (only low byte).

67. All Q3/Q4/Q6a Problems — One-Liner Cheats

Q3 P1 32-bit signed return $-2 \rightarrow R0 = 0xFFFFFFFFFE$.

Q3 P2 64-bit add C-flag test design: low-word sum must cross 2^{32} for one case, not for the other.

Q3 P3 push {r4,r8,r6} on $\text{SP}=0x1020 \rightarrow \text{SP}=0x1014$; R4@1014, R6@1018, R8@101C.

Q3 P4 $A = k \cdot B/8$ in one Op2: factor $k/8 = 1 \pm 1/2^n \rightarrow \text{add/sub } r0,r1, \text{asr } \#n$.

Q3 P5 BIC+ORR splice: $a \&= \sim(15<8)$; $a |= (\&15)<8$;

Q3 P6 $\text{int8_t } 0x81 \rightarrow R1 = 0xFFFFFFFFF81$; $\text{int8_t } 0x7E \rightarrow R0 = 0x7E$.

Q4 P1 if(a=0) ret x else ret y; $\rightarrow \text{cmp } r0,\#0 / \text{blt .else} / \text{mov } r0,r1 / \text{bx lr}$.

Q4 P2 5-bit flags: $(-15)-(-16)$: $C=1$, $V=0$; $16 - 18$: $C=0$, $V=0$.

Q4 P3 [r0,#4]! pre-idx; [r0],#8 post-idx; [r0,r2,ls1 #2] signed scaled ($r2=-1 \rightarrow -4$ bytes).

Q6a P1 LDRD pre-idx 8 + LDRSB sign-ext (0xB0 $\rightarrow 0xFFFFFFFFB0$).

Q6a P2 Saturate $[-5, 5]$: 2 cmp + 2 b.

Q6a P3 Conditional sum: standard array-loop template.

68. Top-20 Single-Instruction Patterns

#	ASM	C
1	add r0,r1,r2	r0=r1+r2
2	adds r0,r1,r2	r0=r1+r2 (sets flags)
3	adc r0,r1,r2	w/ carry-in
4	rsb r0,r1,#0	r0=-r1
5	svn r0,r1	r0=-r1
6	mul r0,r1,r2	r0=r1*r2 (32-bit)
7	mla r0,r1,r2,r3	r0=r3+r1*r2
8	mls r0,r1,r2,r3	r0=r3-ri*r2
9	udiv r0,r1,r2	u div
10	sdiv r0,r1,r2	s div
11	lsl r0,r1,#n	r0=r1<<n
12	asr r0,r1,#n	signed r1>>n
13	lsr r0,r1,#n	unsigned r1>>n
14	add r0,r1,r1,lsl #n	r1 * (2 ⁿ + 1)
15	ldr r0,[r1]	r0=*r1 (word)
16	ldr r0,[r1,r2,lsl #2]	r0=pl word
17	strb r0,[r1],#1	*p++v byte
18	cbz r0,lbl	if(!r0) goto lbl
19	tst r0,#mask	flag-set bit test
20	bic r0,r0,#mask	clear masked bits

69. LDR Pseudo Resolution Detail

ldr Rd,=expr: assembler chooses cheapest encoding by value:

Value range	Emitted insn	Notes
0..0xFF (8-bit)	movs Rd,=expr	16-bit Thumb
0..0xFFFF (16b)	movw Rd,=expr	zeros high 16
big 32-bit	PC-rel ldr + .word	literal pool

When unsure, use ldr Rd,=expr — assembler picks the right encoding. Plain mov Rd,#imm only accepts 8-bit (rotated mod-imm) values.

Building 32-bit literals manually:

movw r0,#0xDEEF @ low 16, zero high
movt r0,#0xDEAD @ high 16, preserve low
@ -> r0 = 0xDEADBEEF

70. Final Exam Strategy

- Read the C prototype FIRST. It dictates LDR variant, signed/unsigned CCS, shift direction.
- Annotate memory layouts. For LDR traces, write bytes at each address BEFORE running through instructions.
- Track register state explicitly. After every instruction, write the changed reg's new value.
- Follow the precedence rules. postfix++ (L1) → * / prefix++ / cast (L2). Walk through the C carefully.
- Pick addressing mode from inc-order. Prefix-inc → pre-idx (!); postfix-inc → post-idx; no inc → basic offset.
- For IT blocks, pick the cond as NL; T slots match, E slots inverse. Cond B ONLY last slot. No CBZ in IT.
- For loops, decide priming vs while(1)-break; CBZ on freshly-loaded bytes.
- Unsigned vs signed. ALWAYS match: shift (ASR/LSR), CCS (GT/HI), load suf (SH/H).
- Don't forget write-back. Prefix ++ on a deref value needs STR after the ADD.
- Sanity-check sign-ext. If MSBit of loaded byte/half = 1, LDRSB/SH produces 0xFFFF-padded high bits.
- Return convention. <32b → R0. 64b → R0:R1. Match return bx lr (leaf) or pop {...,pc} (stem).

71. Debug Pattern Recognition

When tracing student-written code, watch for:

- Missing S-suffix on insn whose flags are tested next → branch tests stale flags from earlier CMP.
- Wrong CCS family (e.g. BLT with uint32_t) → wrong direction near 0x80000000.
- Forgotten write-back after prefix ++ on *p → memory still holds old value.
- LDR offset out of range for byte/halfword → assembler error or reg-offset workaround.
- Misaligned LDRD → requires 8-byte alignment on some cores; Cortex-M4 forgives but slow.
- IT block too long → ITTTTT or 5+ slots → assembler rejects.
- CBZ in IT → assembler rejects.
- Wrong PUSH/POP pairing → stack corruption (e.g. push {r4,lr} but pop {r4,r5,pc}).
- Tail call with stem still on stack → leaks frame.

72. Cortex-M4 Quick Specs

Feature	Value
Word size	32-bit
Instr set	Thumb-2 only (no ARM mode)
Endianness	little (configurable but typ. LE)
Pipeline	3-stage (fetch/decode/execute)
Reg file	R0–R15 (16 GP) + APSR/CONTROL
SP	MSP (handler) + PSP (thread, RTOS)
Bit-banding	SRAM 0x22 region, peripheral 0x42
Vector tbl	0x08000000 default; relocatable
NVIC	240 IRQs, 0–255 priority
SysTick	24-bit downcounter
DSP exts	SIMD, MAC w/ accumulator
FPU	VFPv4-SP single-prec (M4F variant)

73. Bit-Banding (Background)

Direct bit access without read-modify-write:

SRAM bit-band: alias = 0x22000000 + 32 * (addr - 0x20000000) + 4 * bit

Peripheral bit-band: alias = 0x42000000 + 32 * (addr - 0x40000000) + 4 * bit

@ Set bit 5 of word at 0x20000000 atomically:

ldr r0,=0x22000000+0+4*5
mov r1,#1
str r1,[r0]

Reading the alias yields 0 or 1; writing 0/1 sets the bit. Atomic from ISR perspective.

74. When to Use What

Goal	Idiom
test ==0 in reg	cbz (free) or cmp #0/beq
test bit set	tst r,#mask/bne
clear bit	bic r,r,#mask
set bit	orr r,r,#mask
toggle bit	eor r,r,#mask
small unsigned mul/div	lsl/lsr by power-of-2
small signed div	asr (signed)
3*x cheaply	add r,x,x,lsl #1
swap upper/lower halves	ror r,r,#16
abs value	cmp #0 / it lt / rsblt #0
sign mask (-1 or 0)	asr r,r,#31
isolate low N bits	ubfx r,r,#0,#N
sign-ext byte	sxtb r,r (or LDRSB at load)

75. One-Page Summary

Address modes: basic [Ra,#os], pre [Ra,#os]!, post [Ra],#os, reg-off [Ra,Ri,Lsl #n].

Sufs: —/B/SB/H/SH on LOAD; —/B/H on STORE.

C → idx: *p++=post, **p=pre, *(p+k)=basic, p[i]=reg-off.

Sign vs unsigned: CCS GT/HI, shift ASR/LSR, load SH/H. Match!

IT: max 4, T=cond/E=inverse, cond B last only, no CBZ.

Loops: priming-LDRB+CBZ; or while(1)+break.

Stem: push {lr}, pop {pc}.

Modulo: udiv + mls.

64-bit add: adds + adc; sub: subs + sbc.

Sign-ext at LOAD: LDRSB 0x80→0xFFFFFFFF80.

Sign-ext at CALL: int8_t -1 arg → R0=0xFFFFFFFF.

Q-format: encode f * 2ⁿ + neg-wrap; decode TC then /2ⁿ.

PUSH: reg# ascending into addr ascending.